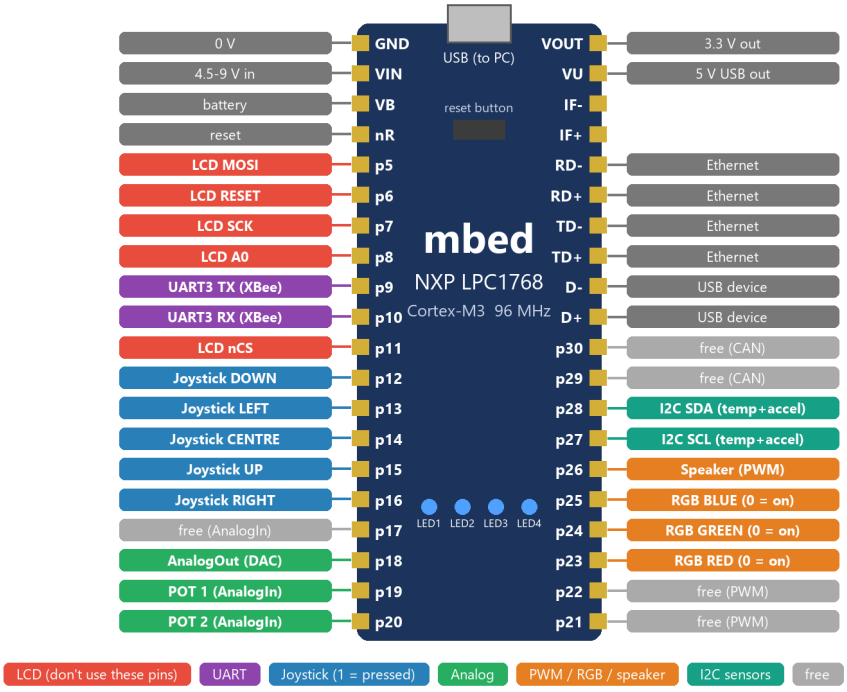
LPC1768 Complete Guide

mbed NXP LPC1768 + mbed Application Board
Mbed Studio | Mbed OS 6 | C++

mbed LPC1768 pins on the Application Board



RBB3023 Embedded & IoT Systems - Test 2 preparation
Chan Li Kai | November 2026

What is inside	
Board, pins, active levels	How to build in Mbed Studio and from the command line
How to flash (copy .bin + reset), serial monitor	Every Mbed OS 6 class with its functions
11 example programs (all compiled), LCD views	Interrupt rules, scheduled programming theory
Worked Test-1-style answer + a practice question	RTOS: threads, mutex, mail, flags, event queue
18 example programs (6 of them RTOS)	Errors, troubleshooting, test-day checklist
16 copy-paste code modules (section 15)	Code_Modules.txt for easy copying

Contents

1. The board at a glance . . . 5

1.1 Pin map of the Application Board . . . 5

2. Software and files . . . 7

2.1 Where everything is . . . 7

2.2 Files inside a program . . . 7

2.3 Making a new program (if you need one) . . . 7

3. How to build in Mbed Studio . . . 8

3.1 Step by step . . . 8

3.2 Red marks in the editor that are NOT errors . . . 9

3.3 Build from the command line (backup) . . . 10

3.4 Reading compile errors . . . 10

4. How to flash (download) to the board . . . 11

4.1 Steps . . . 11

4.2 Flash from the command line (Claude can do this) . . . 12

5. Serial monitor (printf to the PC) . . . 13

6. Program structure and Mbed OS 6 basics . . . 14

6.1 The shape of every program . . . 14

6.2 Time values (std::chrono) . . . 14

6.3 Lecture code (mbed 2) to Mbed OS 6 . . . 14

7. Every class you need (API reference) . . . 16

7.1 DigitalOut - LEDs and any output pin . . . 16

7.2 DigitalIn - joystick and any input pin . . . 16

7.3 InterruptIn - run a function the moment a pin changes . . . 16

7.4 AnalogIn - potentiometers (ADC) . . . 16

7.5 AnalogOut - DAC on p18 . . . 17

7.6 PwmOut - brightness, tones (p21-p26, LED1-LED4) . . . 17

7.7 Timer - measure time (polling) . . . 17

7.8 Ticker - call a function again and again (interrupt) . . . 17

7.9 Timeout - call a function once, later . . . 18

7.10 Waiting . . . 18

7.11 BufferedSerial - UART (PC or another board) . . . 18

7.12 I2C - sensors on p28/p27 . . . 18

7.13 LCD_C12832 - our LCD driver (lcd_c12832.h) . . . 18

7.14 Classes you should NOT use in this course . . . 19

8. Interrupts and scheduled programming . . . 20

8.1 The one rule that crashes the board . . . 20

8.2 Timer vs Ticker (lecture 08a) . . . 20

8.3 Theory points for short answers . . . 20
9. Example programs (all compiled) . . . 21
Example 01 - DigitalOut: running light . . . 21
Example 02 - Timer: two tasks at two rates (lecture 08a Ex 1 + 2) . . . 22
Example 03 - Ticker: interrupts, and changing the speed (lecture 08a Ex 3) . . . 23
Example 04 - Joystick: polling, InterruptIn and Timeout . . . 24
Example 05 - LCD: every drawing function . . . 25
Example 06 - AnalogIn: potentiometers to LCD and PC . . . 26
Example 07 - PwmOut: RGB brightness, speaker, AnalogOut . . . 28
Example 08 - BufferedSerial: commands from the PC . . . 29
Example 09 - UART between two boards . . . 30
Example 10 - I2C temperature and accelerometer (extra) . . . 31
10. Worked example: the Test 1 paper on the LPC1768 . . . 34
10.1 How it is designed . . . 34
10.2 What the LCD shows . . . 36
10.3 Changing it for a different question . . . 36
11. RTOS - Mbed OS threads . . . 37
11.1 Super loop, Ticker or threads? . . . 37
11.2 Words you need (theory) . . . 37
11.3 RTOS API (Mbed OS 6) . . . 38
11.4 Which tool for which job . . . 38
11.5 RTOS rules (read before the examples) . . . 39
Example 12 - Threads: four tasks at four rates . . . 39
Example 13 - Mutex: two threads share one LCD . . . 40
Example 14 - Mail and Queue: passing data . . . 42
Example 15 - EventFlags and Semaphore: waking threads from ISRs . . . 43
Example 16 - EventQueue: interrupts that may printf . . . 45
Example 17 - The Test 1 paper with RTOS threads . . . 46
11.6 RTOS questions for short answers . . . 48
12. Practice question - try it yourself . . . 50
Hints . . . 50
Self-check before you call it done . . . 50
13. Troubleshooting . . . 51
14. Test-day checklist . . . 52
Things that cost marks last time . . . 52
Quick theory self-test . . . 52
15. Code modules (copy and paste) . . . 53
M01 - LED1 - LED4 helpers . . . 54

M02 - RGB LED helper . . . 54
M03 - Joystick: held and pressed-once . . . 55
M04 - LCD: text and numbers . . . 55
M05 - LCD: big number, bars, boxes . . . 56
M06 - Ticker: count every period . . . 57
M07 - Timer: do a task every N ms (changeable speed) . . . 57
M08 - Running LEDs (forward / backward / bounce) . . . 58
M09 - Potentiometers (ADC) . . . 58
M10 - Serial: print to the PC every 0.5 s . . . 59
M11 - Serial: read keys typed on the PC . . . 59
M12 - PWM: RGB brightness and speaker beep . . . 60
M13 - Button interrupt + debounce + Timeout . . . 60
M14 - UART between two boards (text lines) . . . 61
M15 - RTOS: a thread and a mutex . . . 62
M16 - RTOS: EventQueue (interrupt -> printf / LCD) . . . 62
Putting modules together (example 18) . . . 63

1. The board at a glance

The test uses the **mbed NXP LPC1768** module plugged into the **mbed Application Board**. The LPC1768 is the small blue module with the USB socket. The Application Board around it gives you the LCD, joystick, two potentiometers, RGB LED, speaker and two I2C sensors. You program it in C++ with **Mbed OS 6**.

Item	LPC1768 (mbed module)
CPU	ARM Cortex-M3, 96 MHz, ARMv7-M (Thumb-2), 3-stage pipeline, Harvard architecture
Memory	512 KB flash, 64 KB RAM
ADC / DAC	12-bit ADC (6 channels on p15-p20), 10-bit DAC on p18
PWM	6 outputs p21-p26 (+ LED1-LED4); all share ONE period
Serial	4 UARTs: p9/p10, p13/p14, p28/p27 and the USB link (USBTX/USBRX)
Other	2 SPI, 2 I2C, Ethernet, USB device, CAN
Programming	USB drive called MBED (copy a .bin file onto it, press reset)
Tool	Mbed Studio (or the command line build script)

1.1 Pin map of the Application Board

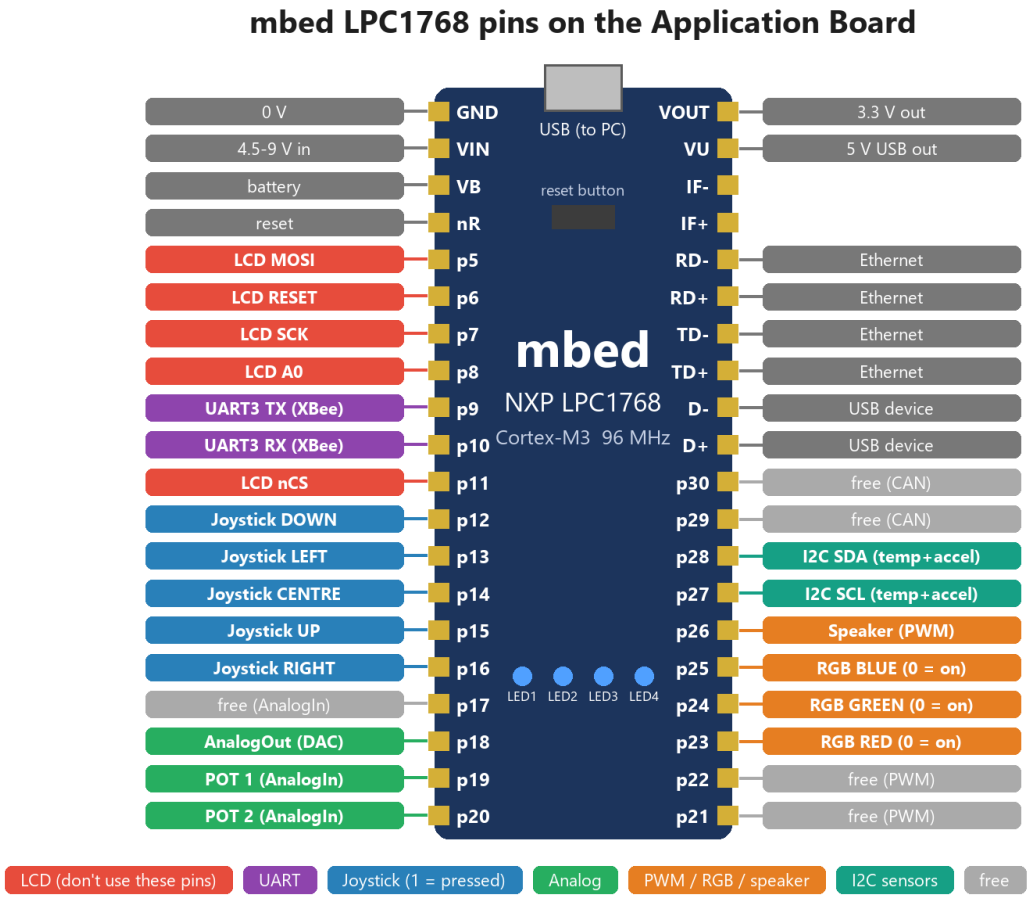


Figure 1 - Every mbed pin and what the Application Board connects to it. Red = LCD: never use p5, p6, p7, p8, p11 for anything else.

Device	Pin(s)	Class to use	Active level / range
LED1 - LED4 (blue, on module)	LED1, LED2, LED3, LED4	DigitalOut / PwmOut	1 = ON
RGB LED	R p23, G p24, B p25	DigitalOut / PwmOut	**0 = ON** (active low)

Device	Pin(s)	Class to use	Active level / range
Joystick	down p12, left p13, centre p14, up p15, right p16	DigitalIn / InterruptIn (PullDown)	1 = pressed
Potentiometer 1 / 2	p19 / p20	AnalogIn	0 - 3.3 V, 12-bit
Speaker	p26	PwmOut	tone = PWM period
LCD 128 x 32 (C12832)	MOSI p5, RESET p6, SCK p7, A0 p8, nCS p11	LCD_C12832 (lcd_c12832.h)	pixel 1 = dark
Temperature LM75B	SDA p28, SCL p27	I2C	address 0x90
Accelerometer MMA7660	SDA p28, SCL p27	I2C	address 0x98
XBee socket / UART3	TX p9, RX p10	BufferedSerial	free for board-to-board
Analog out (DAC)	p18	AnalogOut	0 - 3.3 V
USB serial to PC	USBTX, USBRX	printf / BufferedSerial	115200 baud (mbed_app.json)

Difference from the NUC140 (Test 1 board)

There is **no 7-segment display, no 3x3 keypad and no push button**. The joystick replaces the keypad and buttons, the 128x32 LCD replaces the 7-segment, and LED1-LED4 are the running LEDs. The LCD is only 32 pixels high (NUC140 was 64).

LED1-LED4 are **active high** (1 = on) but the RGB LED is **active low** (0 = on).

2. Software and files

2.1 Where everything is

What	Where
Mbed Studio workspace	C:\Users\TUF FA707RC-HX024W\Mbed Programs
Ready program for the test	Mbed Programs\Test2 (shortcut in D:\...\15 Test 2)
Shared Mbed OS 6 library	C:\Users\TUF FA707RC-HX024W\mbed-os (each program's mbed-os folder is a link to it)
Command line build	Mbed Programs\build_test2.cmd
This guide's examples	D:\...\15 Test 2\Guide_Examples (.cpp) and Guide_Examples\bin (.bin, ready to flash)
Lecture PDFs	D:\...\15 Test 2\Notes (08a Timer/Ticker, 09 ADC end, EIOTS_10 UART end)

2.2 Files inside a program

File	What it does	Change it?
main.cpp	Your program. The only file you normally edit.	Yes
lcd_c12832.h	Our own LCD driver for Mbed OS 6 (text, pixel, box, fill_box, update).	No
font5x7.h	5x7 font used by the LCD driver (from the NUC140 BSP).	No
mbed_app.json	Settings: serial baud 115200 for printf.	Only to change baud
.mbedignore	Skips unused Mbed OS parts (cellular, LoRa, NFC, BLE, WiFi) so builds are faster.	No (never ignore nanostack)
mbed-os (link)	Mbed OS 6.17 source. Shared by all programs.	Never
.mbed, mbed-os.lib	Target LPC1768 and the Mbed OS version.	No
BUILD\LPC1768\ARM C6	Build output: **Test2.bin** is the file you flash.	Generated

mbed_app.json in every program:

```
{
    "target_overrides": {
        "*": {
            "platform.stdio-baud-rate": 115200,
            "platform.stdio-convert-newlines": false
        }
    }
}
```

2.3 Making a new program (if you need one)

1. Easiest: copy the whole **Test2** folder in Mbed Programs, rename the copy, then make a new **mbed-os** link inside it (PowerShell: `New-Item -ItemType Junction -Path <copy>\mbed-os -Target C:\Users\TUF FA707RC-HX024W\mbed-os`). Delete the copied BUILD folder.
2. Or in Mbed Studio: File > New Program > 'empty Mbed OS program', choose **link to an existing shared Mbed OS instance** and pick C:\Users\TUF FA707RC-HX024W\mbed-os. Then copy lcd_c12832.h, font5x7.h, mbed_app.json and .mbedignore from Test2.
3. On test day you don't need a new program: just replace Test2's main.cpp.

3. How to build in Mbed Studio

These screenshots are from Mbed Studio on this laptop with the **Test2** program. The numbers match the steps.

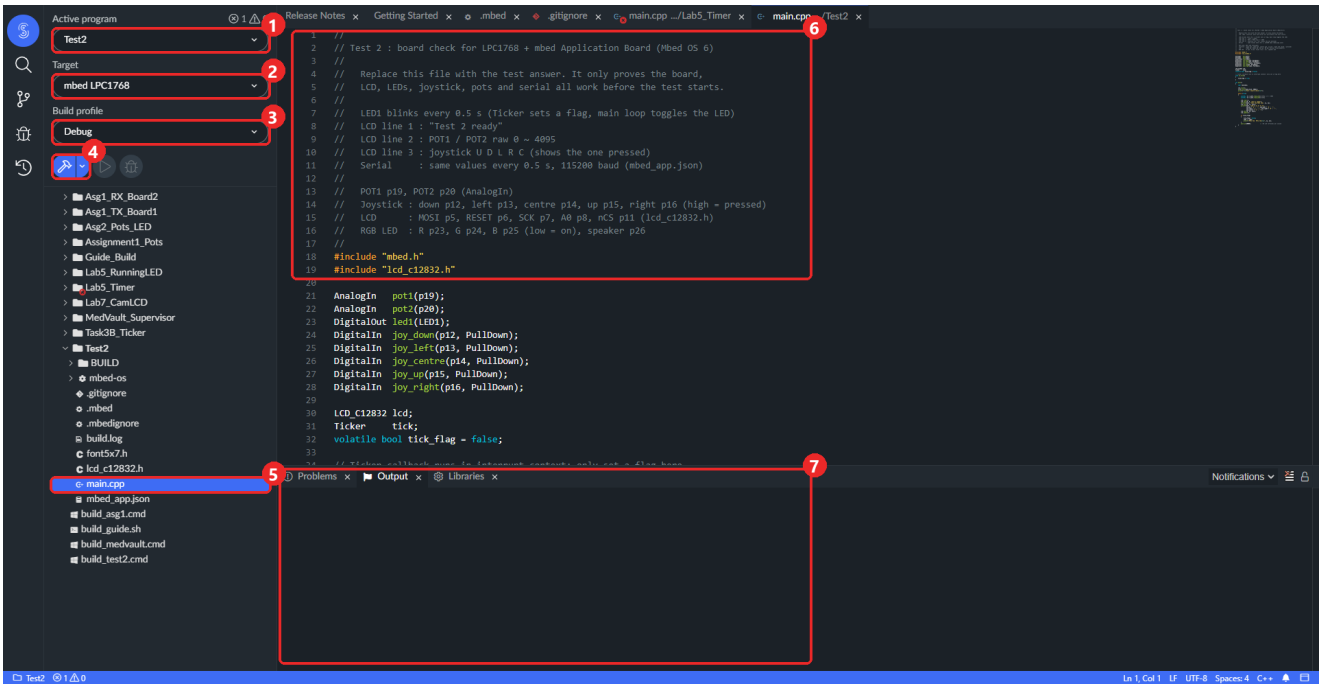


Figure 2 - Mbed Studio main window: (1) Active program, (2) Target, (3) Build profile, (4) Build button, (5) main.cpp in the file tree, (6) code editor, (7) Output panel (build messages).

3.1 Step by step

1. **Active program (1)**: click the box and pick **Test2**. The build button always builds the active program, not the file you are looking at.
2. **Target (2)**: must say **mbed LPC1768**. If it doesn't, click it and type LPC1768 in the search box.
3. **Build profile (3)**: Debug works. Develop is a bit smaller/faster; either is fine for the test.
4. Open **main.cpp (5)** from the tree, edit, press **Ctrl+S** to save.
5. Click the **hammer (4)** to build. Watch the **Output (7)** panel.
6. Success = popup '*Build of Test2 completed: Program compiled*' and the last Output line `Image: BUILD/LPC1768/ARMC6\Test2.bin`.

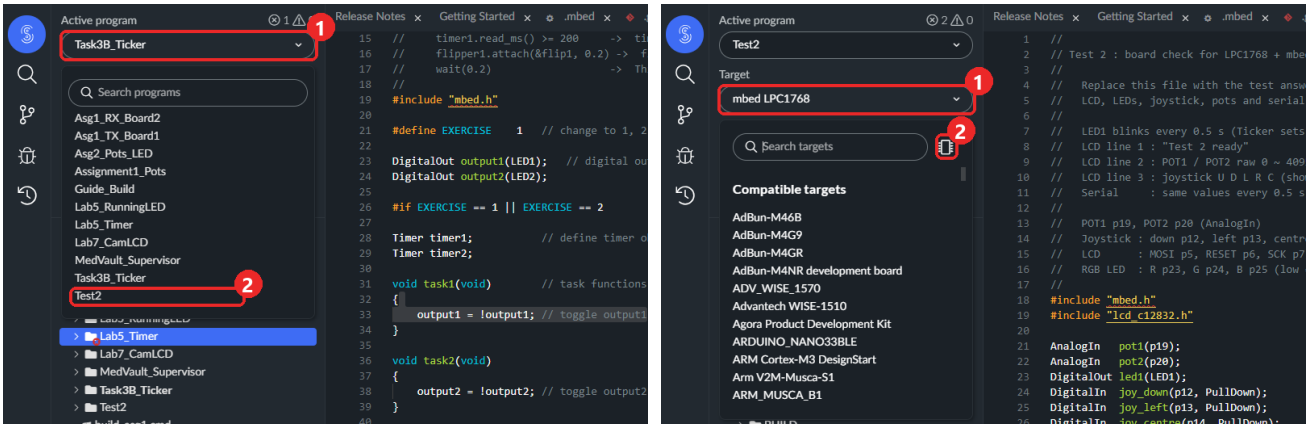


Figure 3 - Left: choose the Active program (1) - pick Test2 (2). Right: the Target list (1); the chip icon (2) detects a connected board.

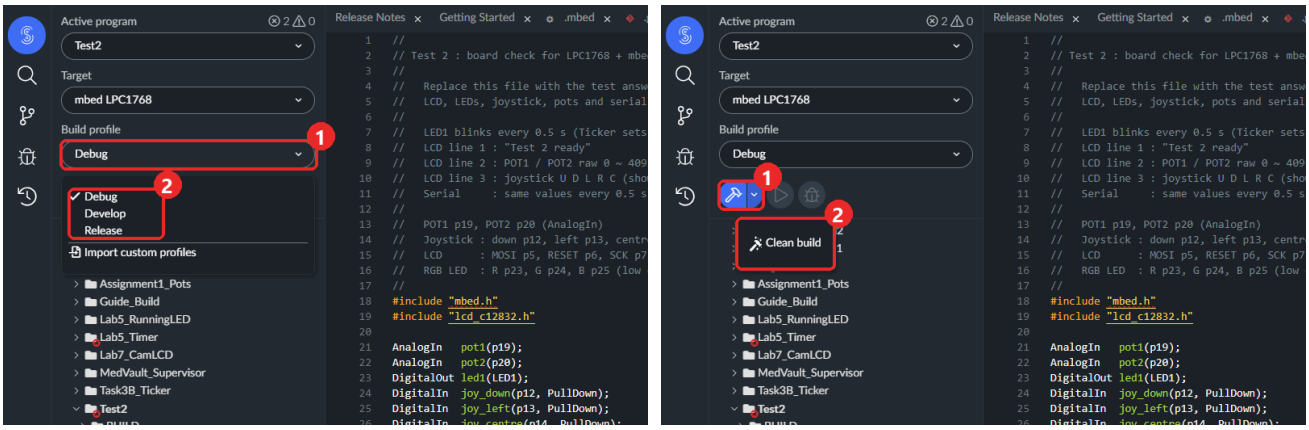


*Figure 4 - Left: Build profile Debug / Develop / Release. Right: the arrow next to the hammer has **Clean build** (use it if a build acts strangely).*

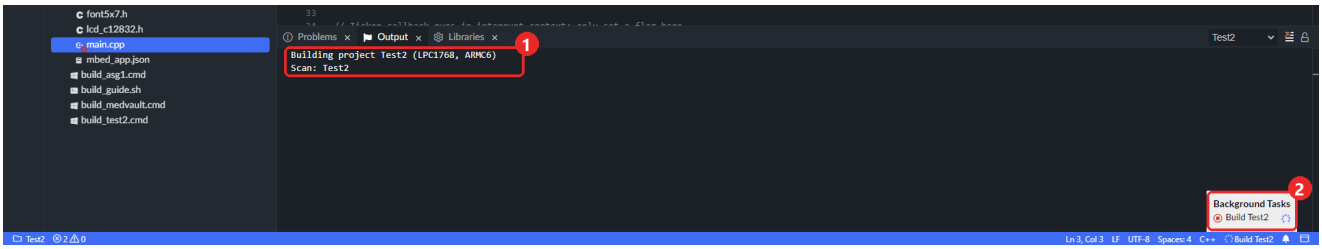


Figure 5 - While building: Output shows 'Building project Test2 (LPC1768, ARMC6)', bottom-right shows the Build task spinning.

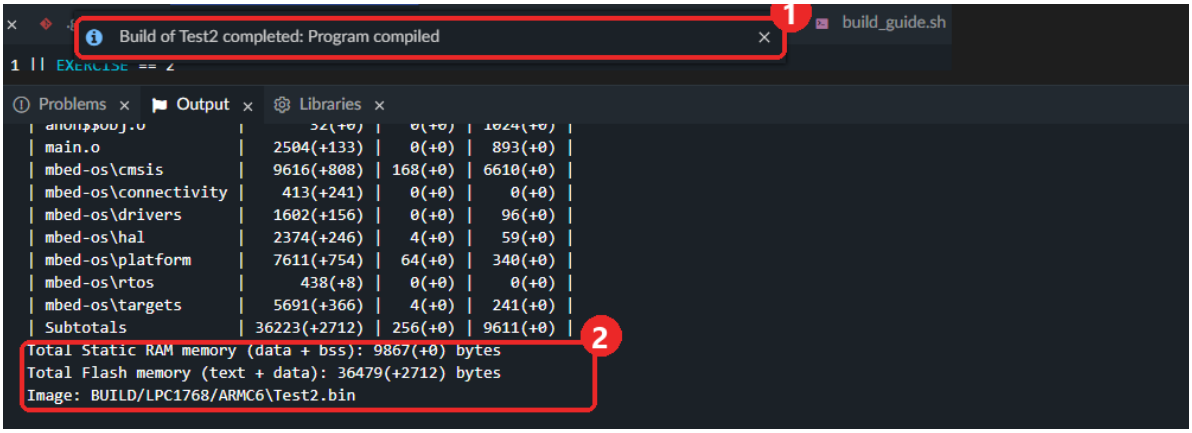


Figure 6 - Build finished: (1) the 'Program compiled' popup, (2) memory use and the .bin path.

> **How long does a build take?**
>
> The first build of a program compiles all of Mbed OS: about **3-5 minutes**. Test2 has already been built, so on test day only main.cpp is compiled again: **about 10-30 seconds**. Changing the Build profile starts a full build again, so pick one and keep it.

3.2 Red marks in the editor that are NOT errors

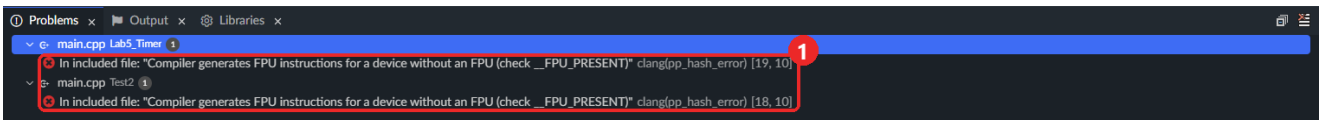


Figure 7 - The Problems tab may show 'Compiler generates FPU instructions for a device without an FPU' on #include "mbed.h".

This message comes from the editor's code checker, not the real compiler. The build in Figure 6 compiled fine with it showing. **Only trust the Output panel of a build.** Red squiggles under `ms` in `200ms` or under Mbed classes are the same thing.

3.3 Build from the command line (backup)

If Mbed Studio misbehaves, the same compiler can be run from a terminal (Claude Code can run it for you):

```
cmd //c "C:\Users\TUF FA707RC-HX024W\Mbed Programs\build_test2.cmd"
```

Command line build (build_test2.cmd) - success

```
> cmd //c "C:\Users\TUF FA707RC-HX024W\Mbed Programs\build_test2.cmd"
Compile [100.0%]: main.cpp
Link: Test2
Elf2Bin: Test2
Total Flash memory (text + data): 33767(+33767) bytes
Image: BUILD/LPC1768/ARMC6\Test2.bin
exit 0
```

Figure 8 - Command line build: 'exit 0' and the Image line mean success. The full log is in Test2\build.log.

3.4 Reading compile errors

Errors look like `main.cpp:10:15: error: ...` = file, **line 10, column 15**. Always fix the **first** error first; later ones are often caused by it. Below are three real errors from the three most common mistakes:

Build output - 3 typical errors (real compiler output)

```
[ERROR] .\main.cpp:10:15: error: expected ';' after expression
                led1 = !led1
                            ^
                            ;
.\main.cpp:11:3: error: reference to 'count' is ambiguous
                count++;
                ^
.\main.cpp:4:5: note: candidate found by name lookup is 'count'
...\include\libcxx\algorithm:1185:1: note: candidate found by name lookup is 'std::count'
.\main.cpp:12:3: error: use of undeclared identifier 'wait'
                wait(0.5);
                ^
3 errors generated.
```

Figure 9 - Missing semicolon (line 10), a global called 'count' (clashes with std::count), and wait() which does not exist in Mbed OS 6.

Error message	Cause	Fix
expected ';' after expression	missing ; at the end of the line above the ^	add the ;
reference to 'count' is ambiguous	global variable named count (std::count exists)	rename: ticks, num, presses ...
use of undeclared identifier 'wait'	Mbed OS 6 removed wait() / wait_ms()	ThisThread::sleep_for(500ms) or wait_us(500000)
no member named 'read_ms'	old mbed 2 Timer API	timer.elapsed_time() >= 200ms
unknown type name 'Serial'	Mbed OS 6 has no Serial class	printf(...) or BufferedSerial
no matching member function for call to 'attach'	attach(&f;, 0.5) with a float, or wrong function type	attach(&f;, 500ms); function must be void f()
invalid operands ... 'std::chrono'	comparing elapsed_time() with a plain int	use 200ms, not 200
L6218E undefined symbol	function declared but not written, or spelt differently	write the function / fix the name

4. How to flash (download) to the board



Figure 10 - The whole cycle. There is no 'download' button needed: the mbed works like a USB stick.

4.1 Steps

1. Plug the mbed into the laptop with the USB cable. A drive appears (on this laptop usually **E:**). It may have no name; it contains the file **MBED.HTM**.
2. Build (section 3). Open the build folder: **Mbed Programs\Test2\BUILD\LPC1768\ARMC6**.
3. Copy **Test2.bin** to the MBED drive (drag and drop, or right-click > Copy, then Paste on the drive). The status LED on the mbed flashes while copying.
4. Press the **reset button** on the mbed module (the small button in the middle of the module). The new program starts.
5. If the board has a COM port, opening a serial terminal and sending a **break** also resets it.

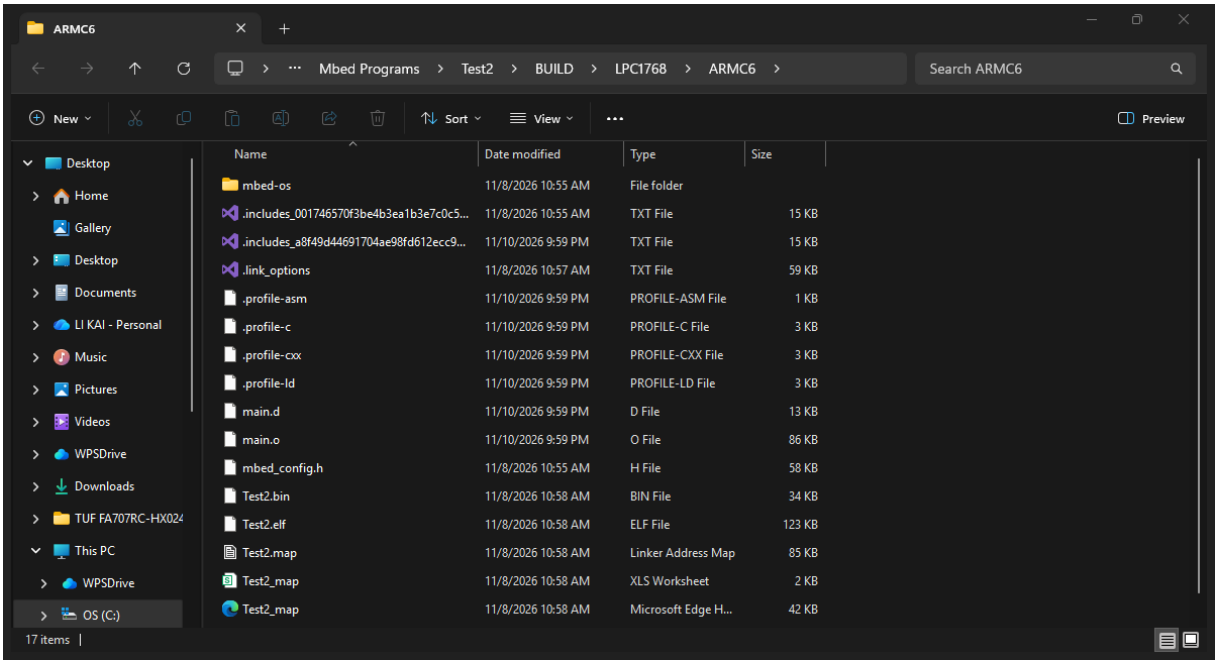


Figure 11 - The build folder in File Explorer. Test2.bin (34 KB) is the file to copy. Ignore .elf/.map.

Rules for the MBED drive

The mbed always runs the **newest .bin** on the drive. Old .bin files can stay, but deleting them avoids confusion.

Copy only the .bin, never the whole folder.

Mbed Studio's Run button (play) may hang at 'waiting for debug probe' on this laptop. Don't wait: copy the .bin by hand.

If nothing changes after reset: check you built the **active** program, check the time stamp of the .bin, copy again, press reset again.

4.2 Flash from the command line (Claude can do this)

```
# Git Bash: find the MBED drive (the one with MBED.HTM), copy the .bin,
# then press reset on the board
BIN="C:/Users/TUF FA707RC-HX024W/Mbed Programs/Test2/BUILD/LPC1768/ARMC6/Test2.bin"
for d in d e f g h; do
    [ -f /$d/MBED.HTM ] && cp "$BIN" /$d/ && echo "copied to $d:"
done
```

5. Serial monitor (printf to the PC)

`printf(...)` sends text over the USB cable to the PC at **115200 baud** (set in mbed_app.json). You need a terminal to see it:

- **Mbed Studio**: when the board is connected, open the Serial Monitor (View menu / the board's serial tab) and set baud 115200.
- **Tera Term / PuTTY**: Serial, port = the mbed's COM port (Device Manager > Ports: 'mbed Serial Port (COMx)', was COM9 on this laptop), 115200, 8 data bits, no parity, 1 stop bit.
- **Python** (pyserial is installed): `py -m serial.tools.miniterm COM9 115200`

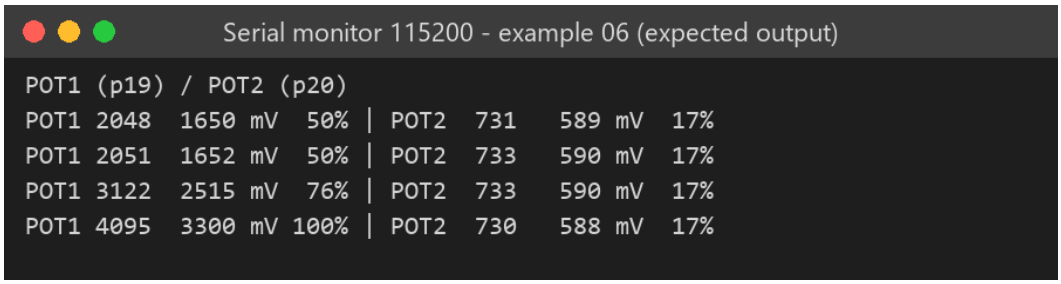


Figure 12 - Expected output of example 06 (pots) in a serial monitor.

printf rules

End lines with `\r\n` (mbed_app.json turns off automatic newline conversion).

%f does not work by default (floats are left out to save space). Print integers: `printf("%d.%02d V", mv/1000, mv%1000/10)`.

Never call printf inside a Ticker / Timeout / InterruptIn function - the board crashes (section 7).

6. Program structure and Mbed OS 6 basics

6.1 The shape of every program

```
//
// header comment: what the program does + which pins (marked in tests)
//
#include "mbed.h"                       // all Mbed classes
#include "lcd_c12832.h"                 // only if you use the LCD

// 1. objects for the hardware (global)
DigitalOut led1(LED1);
DigitalIn  joy_up(p15, PullDown);
LCD_C12832 lcd;
Ticker     tk;

// 2. variables shared with interrupts must be volatile
volatile int ticks = 0;

// 3. interrupt functions: short, only change variables / DigitalOut
void on_tick() { ticks++; }

int main()
{
        // 4. set-up, runs once
        lcd.init();
        tk.attach(&on_tick, 500ms);

        // 5. forever loop: read inputs, update outputs, draw the LCD
        while (true) {
                led1 = joy_up;
                ThisThread::sleep_for(20ms);
        }
}
```

6.2 Time values (std::chrono)

Mbed OS 6 uses time values with units. Write the unit directly after the number:

Write	Means	Used in
500ms	500 milliseconds	attach, sleep_for, elapsed_time() comparison
1s, 2s	seconds	attach(&f;, 1s)
100us	microseconds	attach(&f;, 100us)
std::chrono::milliseconds(x)	a time from a variable x	attach(&f;, std::chrono::milliseconds(delay_ms))
wait_us(1000)	busy wait 1000 us (plain int)	short delays

6.3 Lecture code (mbed 2) to Mbed OS 6

The lecture slides use the old mbed 2 API. It does not compile in Mbed Studio. Translate like this:

Lecture (mbed 2)	Mbed OS 6 (use this)
wait(0.2);	ThisThread::sleep_for(200ms);
wait_ms(200);	ThisThread::sleep_for(200ms);
timer1.read_ms() >= 200	timer1.elapsed_time() >= 200ms
timer1.read()	duration_cast<milliseconds>(timer1.elapsed_time()).count() / 1000.0
flipper.attach(&flip, 0.2);	flipper.attach(&flip, 200ms);

Lecture (mbed 2)	Mbed OS 6 (use this)
flipper.attach_us(&flip, 500);	flipper.attach(&flip, 500us);
Serial pc(USBTX, USBRX); pc.printf(...)	printf(...) (baud in mbed_app.json)
pc.baud(9600); pc.getc(); pc.putc(c);	BufferedSerial pc(USBTX, USBRX, 9600); pc.read(&c,1); pc.write(&c,1);
pc.readable()	pc.readable() (same)
printf("%f", x);	printf("%d", (int)(x * 1000));
DigitalOut out(p5);	DigitalOut out(LED1); (p5 is the LCD)
AnalogIn Ain(PA_0); (Nucleo)	AnalogIn pot1(p19);

7. Every class you need (API reference)

All of these come from `#include "mbed.h"`. 'ISR safe' = allowed inside a Ticker / Timeout / InterruptIn function.

7.1 DigitalOut - LEDs and any output pin

Function	What it does	ISR safe
DigitalOut led(LED1);	create output (starts at 0)	-
DigitalOut g(p24, 1);	create with a start value (1 = RGB green off)	-
led = 1; led.write(1);	set the pin high / low	yes
led = !led;	toggle	yes
int v = led; led.read()	read back what you wrote	yes

7.2 DigitalIn - joystick and any input pin

Function	What it does	ISR safe
DigitalIn up(p15, PullDown);	create input with a pull-down (joystick = 1 when pressed)	-
if (up) ... up.read()	0 or 1	yes
up.mode(PullUp / PullDown / PullNone)	change the pull resistor	yes

Edge detection (one action per press, like `last_key` on the NUC140):

```
bool now = joy_centre;
if (now && !last) { presses++; }        // only on the 0 -> 1 change
last = now;
```

7.3 InterruptIn - run a function the moment a pin changes

Function	What it does
InterruptIn btn(p14);	create (joystick centre)
btn.mode(PullDown);	pull resistor
btn.rise(&on_press);	call on_press when the pin goes 0 -> 1 (pressed)
btn.fall(&on_release);	call on_release when it goes 1 -> 0 (released)
btn.rise(NULL);	stop the interrupt
btn.read()	current level

Joystick contacts bounce, so one press can call the function several times. Ignore calls within 200 ms using a Timer (example 04). On the LPC1768 only port 0 and port 2 pins can interrupt: p5-p18 and p21-p30 can, **p19 and p20 cannot**.

7.4 AnalogIn - potentiometers (ADC)

Function	Returns
AnalogIn pot1(p19);	create (POT1; POT2 = p20; also p15-p18 on other boards)
pot1.read() or float f = pot1;	0.0 - 1.0 (x 3.3 = volts)
pot1.read_u16()	0 - 65535; use >> 4 to get the real 12-bit 0 - 4095
mV = raw * 3300 / 4095	millivolts from the 12-bit value
pct = raw * 100 / 4095	percentage

7.5 AnalogOut - DAC on p18

Function	What it does
AnalogOut dac(p18);	create (only p18)
dac = 0.5f; dac.write(0.5f);	output 0.5 x 3.3 V = 1.65 V
dac.write_u16(32768);	same using 0 - 65535

7.6 PwmOut - brightness, tones (p21-p26, LED1-LED4)

Function	What it does
PwmOut red(p23);	create
red.period_ms(10); red.period(0.01f); red.period_us(100);	PWM period (frequency = 1 / period)
red = 0.25f; red.write(0.25f);	duty cycle 0.0 - 1.0 (25 % high)
red.pulsewidth_ms(1); red.pulsewidth_us(1500);	set high time directly (servo: 1 - 2 ms in 20 ms)
red.read()	current duty

PWM gotchas on the LPC1768

All PwmOut pins share **one period**. Setting the speaker to 440 Hz also changes the RGB period: set it back afterwards (example 07).

RGB is active low: brightness b (0-1) is `red = 1.0f - b;`.

LED1 and p26 (speaker) use the same PWM channel: don't PWM both.

7.7 Timer - measure time (polling)

Function	What it does
Timer t; t.start();	start counting
t.stop(); t.reset();	pause / set back to 0 (keeps running if started)
t.elapsed_time()	time so far (chrono microseconds); compare with 200ms, 1s ...
duration_cast<milliseconds>(t.elapsed_time()).count()	time as a plain number of ms
if (t.elapsed_time() >= 200ms) { task(); t.reset(); }	the scheduled-programming pattern (lecture 08a)

7.8 Ticker - call a function again and again (interrupt)

Function	What it does
Ticker tk;	create
tk.attach(&f, 500ms);	call f every 500 ms, starting 500 ms from now
tk.detach();	stop
tk.detach(); tk.attach(&f, std::chrono::milliseconds(ms));	change the rate

7.9 Timeout - call a function once, later

Function	What it does
Timeout to; to.attach(&f, 2s);	call f once, 2 s from now
to.detach();	cancel

7.10 Waiting

Function	What it does	Inside ISR?
ThisThread::sleep_for(200ms);	sleep, CPU free for other threads (best in main)	NO
wait_us(500);	busy wait in microseconds (int)	only very short
while (t.elapsed_time() < 100ms) {}	busy wait with a Timer (worked on the old board)	no

7.11 BufferedSerial - UART (PC or another board)

Function	What it does
BufferedSerial pc(USBTX, USBRX, 115200);	USB serial to PC
BufferedSerial link(p9, p10, 9600);	UART3 to another board (TX p9, RX p10)
pc.set_blocking(false);	read() returns at once when no data
pc.write(buf, len);	send bytes
n = pc.read(&c, 1);	receive (n = bytes read, -11 = none yet when non-blocking)
pc.readable()	true if data is waiting
pc.set_baud(9600); pc.set_format(8, BufferedSerial::None, 1);	baud, data bits, parity, stop bits

Use **either** printf **or** a BufferedSerial on USBTX/USBRX in one program. Two boards: TX to RX crossed, and **GND to GND**.

7.12 I2C - sensors on p28/p27

Function	What it does
I2C i2c(p28, p27);	SDA, SCL
i2c.frequency(100000);	100 kHz
i2c.write(addr, data, len, repeated);	returns 0 = OK (addr is 8-bit: 0x90 = LM75B)
i2c.read(addr, buf, len);	returns 0 = OK

7.13 LCD_C12832 - our LCD driver (lcd_c12832.h)

Function	What it does
LCD_C12832 lcd;	create (pins are built in)
lcd.init();	once at the start of main
lcd.clear();	clear the buffer (not the screen yet)
lcd.text(x, y, "Hi", scale);	5x7 text; scale 1 = 6x8 px per char (21 chars per line), 2 = 12x16, 3 = 18x24
lcd.pixel(x, y, on);	one pixel (on = false erases)
lcd.box(x, y, w, h);	rectangle outline
lcd.fill_box(x, y, w, h, on);	filled rectangle (on = false erases)

Function	What it does
lcd.update();	send the buffer to the screen (about 0.3 ms)

Screen = 128 x 32 pixels, (0,0) top-left, x to the right (0-127), y down (0-31). Text lines at y = 0, 11, 22 (or 0, 8, 16, 24). **Pattern:** clear, draw everything, update once per loop. Never call update() from a Ticker.

7.14 Classes you should NOT use in this course

- **BusOut / BusIn** - work in main, but crash inside Ticker callbacks (mutex). Use an array `DigitalOut leds[4]` instead.
- **Serial**, **wait()**, **wait_ms()** - removed in Mbed OS 6.
- **Thread / Mutex / EventQueue** - these are the RTOS. They work well but have their own rules: see **section 11** before using them.

8. Interrupts and scheduled programming

8.1 The one rule that crashes the board

> **Inside a Ticker / Timeout / InterruptIn function you may ONLY:**
>
> change `volatile` variables, write DigitalOut / PwmOut, attach/detach Tickers.
>
> **Never**: printf, lcd.update(), sprintf to the LCD, BusOut, sleep_for, long loops, BufferedSerial write.
>
> If you break this, Mbed OS stops with a fatal error: the LEDs flash a pattern (or the program freezes after the first draw).

The safe pattern: the interrupt sets a flag or changes a number, the main loop does the slow work.

```
volatile bool tick_flag = false;
void on_tick() { tick_flag = true; }            // interrupt: fast

int main() {
        tk.attach(&on_tick, 1s);
        while (true) {
                if (tick_flag) {                        // main: slow work
                        tick_flag = false;
                        printf("one second\r\n");
                }
        }
}
```

8.2 Timer vs Ticker (lecture 08a)

	Timer (polling)	Ticker (interrupt)
How	main keeps reading the time; runs the task when it is due	hardware interrupt calls the function at a fixed interval
CPU	busy checking	free for other code in main
Accuracy	depends on how long the main loop takes	exact interval
Risk	a slow part of the loop delays every task	function must be short; no printf/LCD inside
Code	if (t.elapsed_time() >= 200ms) {...; t.reset();}	tk.attach(&f, 200ms);

8.3 Theory points for short answers

- Embedded systems need precise timing (a camera capturing frames at exact intervals) and fast response to events (a crash sensor firing an airbag).
- An **interrupt** halts the running code so a higher-priority routine runs. It can be **timed** (Ticker) or **external / event-driven** (InterruptIn, a mouse click).
- The simplest timer interrupt increments a counter in the background; main runs tasks when the counter reaches a value = **scheduled programming** (some tasks every 10 ms, others every 100 ms).
- Different rates are efficient: engine spark/injection every ~1 ms (up to 8000 rpm), fuel level every ~1000 ms.
- **Challenges**: every task must finish before its period ends or timing overruns and goes out of sync; the amount of code decides the CPU speed needed; tasks due at the same time need priorities (1 ms and 100 ms tasks meet every 100 ms); **no wait/delay (polling) inside scheduled tasks**.
- **Ticker advantage**: no need to read the time, so other code runs while the Ticker calls the function in the background.
- Ticker functions in the lecture: attach (seconds), attach_us (microseconds), member-function versions, detach.

9. Example programs (all compiled)

Each example is a complete main.cpp in **15 Test 2\Guide_Examples**, and its ready .bin is in **Guide_Examples\bin**. To try one: copy its text over Test2's main.cpp, build, flash, or just copy the ready .bin to the MBED drive. All 17 (11 here + 6 RTOS ones in section 11) were compiled for the LPC1768 with Mbed OS 6.17 with **0 errors and 0 warnings in main.cpp**. The LCD pictures are drawn on the PC with the same drawing code as lcd_c12832.h, so they show exactly what the code draws.

#	File	Shows	Size
01	01_digitalout_blink.cpp	DigitalOut array, running light, sleep_for	25 KB
02	02_timer_two_rates.cpp	Timer polling, two tasks at 200 ms / 1 s (lecture Ex 1-2)	27 KB
03	03_ticker_speed.cpp	3 Tickers, change Ticker rate with joystick (lecture Ex 3)	28 KB
04	04_joystick_interrupt_timeout.cpp	DigitalIn edges, InterruptIn + debounce, Timeout	29 KB
05	05_lcd_drawing.cpp	every LCD function, big number, growing bar	32 KB
06	06_analogin_pots_serial.cpp	AnalogIn pots, mV, %, bars, printf	33 KB
07	07_pwm_rgb_speaker.cpp	PwmOut RGB brightness, speaker tones, AnalogOut	30 KB
08	08_serial_pc_commands.cpp	BufferedSerial: commands from the PC	31 KB
09	09_uart_two_boards.cpp	UART3 p9/p10 between two boards	36 KB
10	10_i2c_temp_accel.cpp	I2C LM75B temperature + MMA7660 tilt	34 KB
11	11_worked_example_test1_style.cpp	Test 1 paper redone on this board (section 10)	33 KB
12	12_rtos_threads.cpp	RTOS: 3 threads + main at different rates (section 11)	35 KB
13	13_rtos_mutex_lcd.cpp	RTOS: Mutex - two threads share the LCD	35 KB
14	14_rtos_mail_queue.cpp	RTOS: Mail between threads, Queue from an ISR	37 KB
15	15_rtos_eventflags_semaphore.cpp	RTOS: EventFlags + Semaphore woken by ISRs	37 KB
16	16_rtos_eventqueue.cpp	RTOS: EventQueue - printf/LCD work posted from ISRs	38 KB
17	17_rtos_worked_example.cpp	RTOS: the Test 1 paper with one thread per task	35 KB
18	18_modules_combined.cpp	5 code modules joined: pot speed + LCD (section 15)	33 KB

Example 01 - DigitalOut: running light

- An **array of DigitalOut** lets you use a loop and an index, like the 4 LEDs on the NUC140.
- On the board: one LED moves LED1 to LED4 and back, 200 ms per step.

```
//
// Example 01 : DigitalOut - blink and running light on LED1 ~ LED4
//
//   LED1 ~ LED4 are the 4 blue LEDs on the mbed module (1 = on)
//   One LED at a time moves LED1 -> LED4 and back, 200 ms per step
//
#include "mbed.h"

DigitalOut leds[4] = { DigitalOut(LED1), DigitalOut(LED2),
                       DigitalOut(LED3), DigitalOut(LED4) };

// turn on only LED number n (0 ~ 3)
void show_one(int n)
{
        for (int i = 0; i < 4; i++)
                leds[i] = (i == n);
}

int main()
{
        int pos = 0, dir = 1;
```

```
        while (true)
        {
                show_one(pos);
                ThisThread::sleep_for(200ms);   // Mbed OS 6: no wait(0.2)

                pos += dir;
                if (pos == 3 || pos == 0)       // bounce at both ends
                        dir = -dir;
        }
}
```

> **Try this**
>
> Light two LEDs at a time. Change 200ms to 100ms. Make it go only one way (LED1 to LED4, then start again).

Example 02 - Timer: two tasks at two rates (lecture 08a Ex 1 + 2)

- Same structure as the lecture slide, translated to Mbed OS 6 and moved from p5/p6 to LED1/LED2.
- On the board: LED1 blinks fast (toggles every 0.2 s), LED2 slow (every 1 s).

```
//
// Example 02 : Timer - scheduled programming by polling (lecture 08a, Exercise 1 + 2)
//
//   task1 toggles LED1 every 200 ms  (timer1)
//   task2 toggles LED2 every 1000 ms (timer2)
//   The main loop keeps reading both timers and runs a task when it is due.
//
//   Lecture (mbed 2)         ->  Mbed OS 6
//   timer1.read_ms() >= 200  ->  timer1.elapsed_time() >= 200ms
//   DigitalOut output1(p5)   ->  LED1  (p5 is the LCD on the app board)
//
#include "mbed.h"

Timer timer1, timer2;           // define timer objects
DigitalOut output1(LED1);
DigitalOut output2(LED2);

void task1(void);               // task function prototypes
void task2(void);

int main()
{
        timer1.start();         // start timers counting
        timer2.start();

        while (true)
        {
                if (timer1.elapsed_time() >= 200ms)     // read time
                {
                        task1();
                        timer1.reset();                 // back to 0
                }
                if (timer2.elapsed_time() >= 1000ms)
                {
                        task2();
                        timer2.reset();
                }
        }
}

void task1(void)
{
        output1 = !output1;     // toggle LED1
}

void task2(void)
{
        output2 = !output2;     // toggle LED2
}
```

> **Try this**
>
> Add task3 on LED3 every 500 ms with a third Timer. What happens if you put ThisThread::sleep_for(300ms) inside task1? (Both rates go wrong - that is the 'no delays in scheduled programs' rule.)

Example 03 - Ticker: interrupts, and changing the speed (lecture 08a Ex 3)

- Three Tickers run at the same time. main() is free; it only watches the joystick.
- To change a Ticker's speed: **detach()** then **attach()** with the new time.
- The joystick uses **edge detection**, so holding it doesn't keep changing the speed.

```
//
// Example 03 : Ticker - interrupt-driven timing (lecture 08a, Exercise 3)
//
//   flipper1 toggles LED1 every 200 ms, flipper2 toggles LED2 every 1 s
//   flipper3 runs a light along LED3 -> LED4, its speed is changed with
//   the joystick: up = slower (+50 ms), down = faster (-50 ms), 50 ~ 1000 ms
//   Serial (115200) prints the new speed after each change.
//
//   RULE: a Ticker callback runs in interrupt context.
//         Only change DigitalOut / volatile variables there.
//         printf, LCD update, BusOut, mutexes -> do them in main().
//
//   Joystick : up p15, down p12 (1 = pressed)
//
#include "mbed.h"

Ticker flipper1, flipper2, flipper3;
DigitalOut led1(LED1), led2(LED2), led3(LED3), led4(LED4);
DigitalIn  joy_up(p15, PullDown);
DigitalIn  joy_down(p12, PullDown);

volatile int step_ms = 300;             // flipper3 period, changed by the joystick

void flip1() { led1 = !led1; }
void flip2() { led2 = !led2; }
void flip3()                            // LED3 and LED4 take turns
{
        led3 = !led3;
        led4 = !led3;
}

// detach + attach is how the period of a Ticker is changed
void set_speed(int ms)
{
        flipper3.detach();
        flipper3.attach(&flip3, std::chrono::milliseconds(ms));
}

int main()
{
        bool last_up = false, last_down = false;

        led1 = 0;
        led2 = 0;
        flipper1.attach(&flip1, 200ms);         // lecture: attach(&flip1, 0.2)
        flipper2.attach(&flip2, 1s);            // lecture: attach(&flip2, 1.0)
        set_speed(step_ms);
        printf("\r\nTicker demo, LED3/LED4 step = %d ms\r\n", step_ms);

        while (true)                            // main is free: the Tickers interrupt it
        {
                bool up = joy_up, down = joy_down;

                if (up && !last_up && step_ms < 1000)           // edge = once per press
                {
                        step_ms += 50;
                        set_speed(step_ms);
                        printf("slower: %d ms\r\n", step_ms);
                }
```

```
                if (down && !last_down && step_ms > 50)
                {
                        step_ms -= 50;
                        set_speed(step_ms);
                        printf("faster: %d ms\r\n", step_ms);
                }
                last_up = up;
                last_down = down;
                ThisThread::sleep_for(20ms);            // also gives ~20 ms debounce
        }
}
```

Serial monitor 115200 - example 03 (expected output)

```
Ticker demo, LED3/LED4 step = 300 ms
slower: 350 ms
slower: 400 ms
faster: 350 ms
faster: 300 ms
faster: 250 ms
```

Serial output while pressing joystick up twice and down three times.

> **Try this**
>
> Make joystick left stop flipper2 (detach) and right start it again (attach).

Example 04 - Joystick: polling, InterruptIn and Timeout

- Left/right are **polled** in main (DigitalIn). Centre uses an **interrupt** (InterruptIn.rise).
- The interrupt only changes variables and the RGB pin; printf happens in main via `pressed_flag`.
- Timeout = one-shot: the green LED goes off 1 s after the last press.

```
//
// Example 04 : Joystick - DigitalIn polling, InterruptIn and Timeout
//
//   Polling (DigitalIn) : left / right move a lit LED along LED1 ~ LED4
//   Interrupt (InterruptIn) : centre press counts presses (debounced 200 ms)
//   Timeout : after a centre press the green RGB LED stays on for 1 s
//   Serial (115200) prints every press
//
//   Joystick : down p12, left p13, centre p14, up p15, right p16 (1 = pressed)
//   RGB LED  : green p24 (0 = on)
//
#include "mbed.h"

DigitalOut  leds[4] = { DigitalOut(LED1), DigitalOut(LED2),
                        DigitalOut(LED3), DigitalOut(LED4) };
DigitalOut  green(p24, 1);              // start off (active low)
DigitalIn   joy_left(p13, PullDown);
DigitalIn   joy_right(p16, PullDown);
InterruptIn joy_centre(p14);
Timeout     green_off;
Timer       debounce;

volatile int  presses = 0;
volatile bool pressed_flag = false;

void led_off()                          // Timeout callback (interrupt context)
{
        green = 1;
}

void on_centre()                        // InterruptIn callback (interrupt context)
{
        if (debounce.elapsed_time() < 200ms)    // ignore switch bounce
                return;
        debounce.reset();
        presses++;
```

```
        pressed_flag = true;            // main() does the printf
        green = 0;                      // green on ...
        green_off.attach(&led_off, 1s); // ... and off again in 1 s
}

int main()
{
        int pos = 0;
        bool last_left = false, last_right = false;

        debounce.start();
        joy_centre.mode(PullDown);
        joy_centre.rise(&on_centre);    // rise = pressed, fall = released

        while (true)
        {
                bool l = joy_left, r = joy_right;

                if (l && !last_left && pos > 0)  pos--; // edge detection
                if (r && !last_right && pos < 3) pos++;
                last_left = l;
                last_right = r;

                for (int i = 0; i < 4; i++)
                        leds[i] = (i == pos);

                if (pressed_flag)
                {
                        pressed_flag = false;
                        printf("centre pressed %d times\r\n", presses);
                }
                ThisThread::sleep_for(20ms);
        }
}
```

> **Try this**
>
> Count presses of joystick up with another InterruptIn and show both counts with printf.

Example 05 - LCD: every drawing function

- Always: **clear() - draw - update()**. Drawing goes into memory; update() sends it to the screen.
- A bar that grows **up**: `fill_box(x, 32 - h, w, h)`. A bar that grows to the right: `fill_box(x, y, w, h)` with w growing.

```
//
// Example 05 : LCD 128 x 32 (C12832) with lcd_c12832.h
//
//   Shows every drawing function:
//     text (scale 1 and 2), pixel, box (outline), fill_box (filled / erase)
//   Layout : big counter 00~99 on the left (scale 2, +1 every 0.5 s),
//            a bar that grows up on the right side,
//            small text + 3 boxes in the middle
//
//   LCD pins : MOSI p5, RESET p6, SCK p7, A0 p8, nCS p11
//   Coordinates : x 0 ~ 127 left -> right, y 0 ~ 31 top -> bottom
//
#include "mbed.h"
#include "lcd_c12832.h"

LCD_C12832 lcd;

int main()
{
        char text[24];
        int  num = 0;

        lcd.init();                             // once, at the start

        while (true)
```

```
        {
                lcd.clear();                    // 1. clear the buffer

                sprintf(text, "%02d", num);     // 2. draw into the buffer
                lcd.text(0, 0, text, 2);        //    scale 2 = 12 x 16 px per char
                lcd.text(0, 20, "count");       //    scale 1 = 6 x 8 px per char

                lcd.text(40, 0, "box  fill  dot");
                lcd.box(40, 12, 14, 14);                // outline
                lcd.fill_box(70, 12, 14, 14);           // filled
                lcd.fill_box(74, 16, 6, 6, false);      // erase a hole in it
                lcd.pixel(101, 18);                     // one dot
                lcd.pixel(102, 18);

                int h = num % 33;                       // bar height 0 ~ 32
                lcd.box(116, 0, 12, 32);                // bar frame
                lcd.fill_box(118, 32 - h, 8, h);        // grows upward

                lcd.update();                   // 3. send the buffer to the LCD

                num = (num + 1) % 100;
                ThisThread::sleep_for(500ms);
        }
}
```

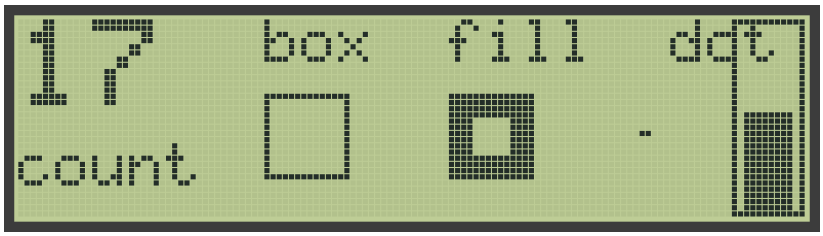


Figure - LCD of example 05 when the counter is 17: big number (scale 2), outline box, filled box with a hole, two pixels, bar on the right 17 px high.

> **Try this**
>
> Draw a box that moves left to right across the screen and bounces back (change x each loop, flip direction at 0 and 127-w).

Example 06 - AnalogIn: potentiometers to LCD and PC

- 8 readings averaged = steady numbers. 12-bit = 0 to 4095.
- Volts printed as integers (`mv / 1000` and `mv % 1000 / 10`) because %f is off.

```
//
// Example 06 : AnalogIn (ADC) - POT1 / POT2 to the LCD and the PC serial monitor
//
//   POT1 p19, POT2 p20 : 12-bit ADC, 0 ~ 3.3 V
//     read()      -> 0.0 ~ 1.0
//     read_u16()  -> 0 ~ 65535, >> 4 = 0 ~ 4095 (the real 12 bits)
//     millivolts  = raw * 3300 / 4095
//   Serial : one line every 0.5 s, 115200 baud (mbed_app.json)
//     POT1 2048  1650 mV  50% | POT2  410   330 mV  10%
//   LCD : value, voltage, % and a bar for each pot
//
#include "mbed.h"
#include "lcd_c12832.h"

AnalogIn   pot1(p19);
AnalogIn   pot2(p20);
LCD_C12832 lcd;

// average of 8 readings so the value does not jitter, 0 ~ 4095
int read_pot(AnalogIn &pot)
{
        int sum = 0;

        for (int i = 0; i < 8; i++)
```

```
                sum += pot.read_u16() >> 4;
        return sum / 8;
}

// one pot: text line at y, bar under it
void show_pot(int y, int num, int raw)
{
        char text[24];
        int  mv  = raw * 3300 / 4095;
        int  pct = raw * 100 / 4095;

        // printf / sprintf have no %f by default -> print volts as V.vv with integers
        sprintf(text, "POT%d %4d %d.%02dV %3d%%", num, raw, mv / 1000, mv % 1000 / 10, pct);
        lcd.text(0, y, text);
        lcd.box(0, y + 9, 128, 6);
        lcd.fill_box(1, y + 10, raw * 126 / 4095, 4);
}

int main()
{
        lcd.init();
        printf("\r\nPOT1 (p19) / POT2 (p20)\r\n");

        while (true)
        {
                int v1 = read_pot(pot1);
                int v2 = read_pot(pot2);

                lcd.clear();
                show_pot(0, 1, v1);
                show_pot(16, 2, v2);
                lcd.update();

                printf("POT1 %4d %5d mV %3d%% | POT2 %4d %5d mV %3d%%\r\n",
                       v1, v1 * 3300 / 4095, v1 * 100 / 4095,
                       v2, v2 * 3300 / 4095, v2 * 100 / 4095);
                ThisThread::sleep_for(500ms);
        }
}
```

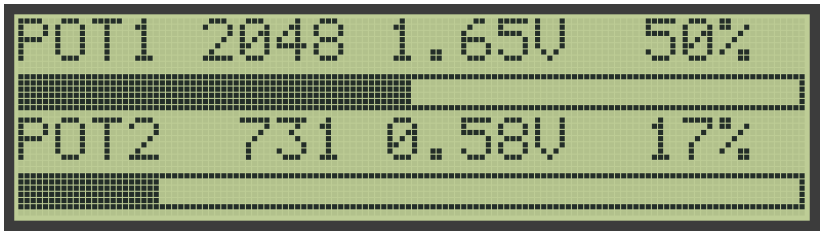


Figure - LCD of example 06 with POT1 at the middle (2048) and POT2 low (731).

Serial monitor 115200 - example 06 (expected output)

```
POT1 (p19) / POT2 (p20)
POT1 2048  1650 mV  50% | POT2  731   589 mV  17%
POT1 2051  1652 mV  50% | POT2  733   590 mV  17%
POT1 3122  2515 mV  76% | POT2  733   590 mV  17%
POT1 4095  3300 mV 100% | POT2  730   588 mV  17%
```

Serial output every 0.5 s.

Try this

Light 1-4 LEDs depending on POT1 % (<25, <50, <75, else) - that was Assignment 2.

Example 07 - PwmOut: RGB brightness, speaker, AnalogOut

- Turn POT1: red gets brighter. POT2: blue. Joystick up/down: short beep at 440 / 880 Hz.
- A tone = PWM with period 1/f and duty 0.5.

```
//
// Example 07 : PwmOut - RGB LED brightness, speaker tone, AnalogOut
//
//   POT1 sets the red brightness, POT2 the blue brightness (RGB is active low)
//   Joystick up = 440 Hz beep for 150 ms on the speaker, down = 880 Hz
//   AnalogOut p18 outputs the POT2 voltage (0 ~ 3.3 V, 10-bit DAC)
//
//   RGB : R p23, G p24, B p25 (0 = on)   Speaker : p26
//   PWM pins on the LPC1768 : p21 ~ p26 (+ LED1 ~ LED4)
//   GOTCHA : all PwmOut pins share ONE period (one PWM timer), so the
//            speaker's period also changes the RGB period -> set it back.
//            LED1 uses the same PWM channel as p26 (speaker): don't use both.
//
#include "mbed.h"

PwmOut    red(p23), green(p24), blue(p25);
PwmOut    speaker(p26);
AnalogIn  pot1(p19), pot2(p20);
AnalogOut dac(p18);
DigitalIn joy_up(p15, PullDown);
DigitalIn joy_down(p12, PullDown);

// play a tone of freq Hz for ms milliseconds
void beep(int freq, int ms)
{
        speaker.period(1.0f / freq);    // period = 1 / frequency
        speaker = 0.5f;                 // 50 % duty = loudest square wave
        ThisThread::sleep_for(std::chrono::milliseconds(ms));
        speaker = 0.0f;                 // silent
        red.period_ms(10);              // shared period: back to 100 Hz for the RGB
}

int main()
{
        red.period_ms(10);              // 100 Hz PWM, no visible flicker
        green.period_ms(10);
        blue.period_ms(10);
        green = 1.0f;                   // 1.0 = green fully off (active low)

        while (true)
        {
                float a = pot1.read();  // 0.0 ~ 1.0
                float b = pot2.read();

                red  = 1.0f - a;        // active low: invert
                blue = 1.0f - b;
                dac  = b;               // 0.0 ~ 1.0 -> 0 ~ 3.3 V on p18

                if (joy_up)   beep(440, 150);
                if (joy_down) beep(880, 150);
                ThisThread::sleep_for(20ms);
        }
}
```

> **Try this**
>
> Make POT2 control the beep pitch (200-2000 Hz). Play a 3-note tune on joystick centre.

Example 08 - BufferedSerial: commands from the PC

- Non-blocking read: the loop never stops waiting for the PC, so the 2 s 'tick' keeps going.
- This program does not use printf because it owns USBTX/USBRX with BufferedSerial.

```
//
// Example 08 : BufferedSerial - talk to the PC (Tera Term / Mbed Studio serial monitor)
//
//   USB serial (USBTX / USBRX) at 115200 baud
//   Type on the PC:  1 ~ 4 = toggle LED1 ~ LED4,  s = status,  anything else = echoed
//   The board also sends "tick n" every 2 s (Timer, no blocking)
//
//   Lecture (mbed 2): Serial pc(USBTX, USBRX); pc.printf(...); pc.getc(); pc.readable()
//   Mbed OS 6 has no Serial class -> BufferedSerial read() / write()
//   Do not mix printf and a BufferedSerial on the same USBTX/USBRX: use one of them.
//
#include "mbed.h"

BufferedSerial pc(USBTX, USBRX, 115200);
DigitalOut leds[4] = { DigitalOut(LED1), DigitalOut(LED2),
                       DigitalOut(LED3), DigitalOut(LED4) };
Timer t;

// send a C string
void send(const char *s)
{
        pc.write(s, strlen(s));
}

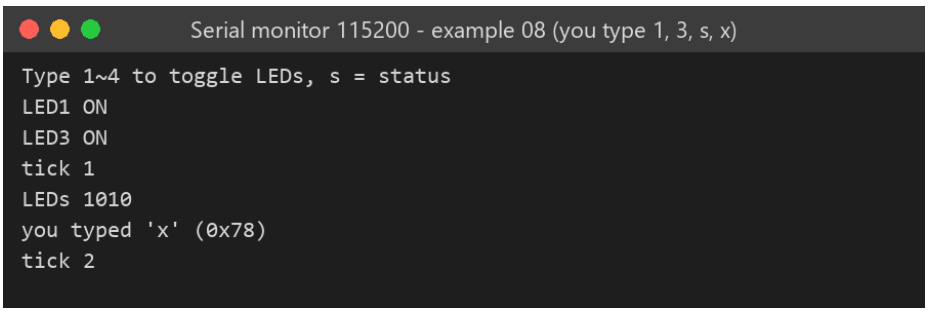
int main()
{
        char line[48];
        int  ticks = 0;

        pc.set_blocking(false);         // read() returns at once if nothing came
        t.start();
        send("\r\nType 1~4 to toggle LEDs, s = status\r\n");

        while (true)
        {
                char c;

                if (pc.read(&c, 1) == 1)        // one byte received
                {
                        if (c >= '1' && c <= '4')
                        {
                                int n = c - '1';
                                leds[n] = !leds[n];
                                sprintf(line, "LED%d %s\r\n", n + 1, leds[n] ? "ON" : "OFF");
                                send(line);
                        }
                        else if (c == 's')
                        {
                                sprintf(line, "LEDs %d%d%d%d\r\n",
                                        leds[0].read(), leds[1].read(), leds[2].read(), leds[3].read());
                                send(line);
                        }
                        else
                        {
                                sprintf(line, "you typed '%c' (0x%02X)\r\n", c, c);
                                send(line);
                        }
                }

                if (t.elapsed_time() >= 2s)
                {
                        t.reset();
                        sprintf(line, "tick %d\r\n", ++ticks);
                        send(line);
                }
        }
}
```



Typing 1, 3, s and x in the serial monitor.

> **Try this**
>
> Add command 'r' that resets all LEDs, and 'p' that sends the POT1 value.

Example 09 - UART between two boards

- Same program on both boards. Wires: p9 to the other p10, p10 to the other p9, GND to GND.
- Messages are text lines ending in \n, so the receiver knows where each message ends.
- If RX stays '(nothing yet)': check TX/RX are crossed, GND is joined, both use 9600.

```
//
// Example 09 : UART between two LPC1768 boards (lecture 10, "two mbed" circuit)
//
//   The SAME program goes on both boards.
//   Wiring : board A p9 (TX) -> board B p10 (RX)
//            board A p10 (RX) <- board B p9 (TX)
//            GND <-> GND  (always join the grounds)
//   9600 baud, one line "P1=nnnn\n" every 0.5 s with this board's POT1
//   LCD line 1 = what this board sends, line 2 = what it received
//   Joystick centre on one board lights LED1 on the other board ("B1\n" / "B0\n")
//
//   p9/p10 = UART3 (XBee socket pins), the only free UART on the app board
//   (p13/p14 = joystick, p27/p28 = I2C sensors)
//
#include "mbed.h"
#include "lcd_c12832.h"

BufferedSerial link(p9, p10, 9600);     // TX, RX, baud
AnalogIn   pot1(p19);
DigitalIn  joy_centre(p14, PullDown);
DigitalOut led1(LED1);
LCD_C12832 lcd;
Timer t;

int main()
{
        char out[24], rx_line[24] = "", show[24] = "(nothing yet)";
        int  rx_len = 0;
        bool last_btn = false;

        lcd.init();
        link.set_blocking(false);
        t.start();

        while (true)
        {
                // send POT1 every 0.5 s
                if (t.elapsed_time() >= 500ms)
                {
                        t.reset();
                        int len = sprintf(out, "P1=%d\n", pot1.read_u16() >> 4);
                        link.write(out, len);
                }

                // send the button state when it changes
                bool btn = joy_centre;
                if (btn != last_btn)
```

```
                {
                        link.write(btn ? "B1\n" : "B0\n", 3);
                        last_btn = btn;
                }

                // collect received bytes into a line, act on '\n'
                char c;
                while (link.read(&c, 1) == 1)
                {
                        if (c == '\n')
                        {
                                rx_line[rx_len] = 0;
                                if (rx_line[0] == 'B')          // button message
                                        led1 = (rx_line[1] == '1');
                                else
                                        strcpy(show, rx_line);  // POT1 message
                                rx_len = 0;
                        }
                        else if (rx_len < 20)
                                rx_line[rx_len++] = c;
                }

                lcd.clear();
                lcd.text(0, 0, "TX:");
                lcd.text(24, 0, out);
                lcd.text(0, 12, "RX:");
                lcd.text(24, 12, show);
                lcd.text(0, 24, "p9 TX  p10 RX  9600");
                lcd.update();
                ThisThread::sleep_for(20ms);
        }
}
```

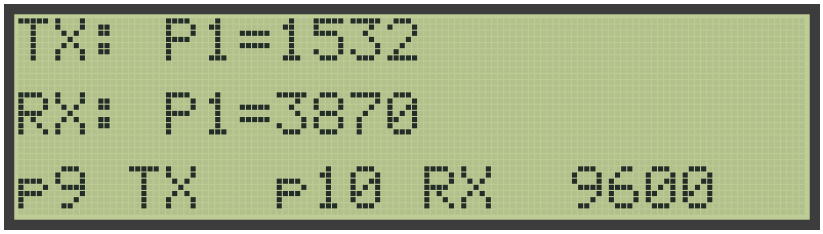


Figure - LCD of example 09: this board sends P1=1532, it received P1=3870 from the other board.

> **Try this**
>
> Send the joystick direction as a letter (U/D/L/R) and show it on the other board's LCD.

Example 10 - I2C temperature and accelerometer (extra)

- Not in the lectures, but the sensors are on the board - useful for a bonus or the project.
- Addresses in Mbed are 8-bit (7-bit address shifted left): LM75B 0x48 to 0x90, MMA7660 0x4C to 0x98.

```
//
// Example 10 : I2C sensors on the app board (extra - not in the lectures)
//
//   LM75B temperature  : 8-bit address 0x90, register 0 = 2 bytes
//                        t = ((b0 << 8) | b1) >> 5 (signed 11-bit), 0.125 C per count
//   MMA7660 accelerometer : 8-bit address 0x98
//                        write {0x07, 0x01} = active mode
//                        registers 0, 1, 2 = X, Y, Z, 6-bit signed, ~21 counts per g
//   I2C pins : SDA p28, SCL p27
//   LCD shows the temperature and X/Y/Z, serial prints them every 0.5 s
//
#include "mbed.h"
#include "lcd_c12832.h"

#define LM75B_ADDR      0x90
#define MMA7660_ADDR    0x98

I2C        i2c(p28, p27);               // SDA, SCL
```

```
LCD_C12832 lcd;

// temperature in tenths of a degree (e.g. 253 = 25.3 C)
int read_temp_x10()
{
        char reg = 0, d[2];

        i2c.write(LM75B_ADDR, &reg, 1, true);   // true = repeated start, keep the bus
        i2c.read(LM75B_ADDR, d, 2);
        int16_t raw = (int16_t)((d[0] << 8) | d[1]) >> 5;       // keeps the sign
        return raw * 10 / 8;                            // 0.125 C = 1/8
}

// 6-bit two's complement -> -32 ~ +31
int six_bit(char v)
{
        v &= 0x3F;
        return (v & 0x20) ? v - 64 : v;
}

int main()
{
        char text[24];
        char cmd[2] = { 0x07, 0x01 };           // MODE register = active

        lcd.init();
        i2c.frequency(100000);
        i2c.write(MMA7660_ADDR, cmd, 2);

        while (true)
        {
                char reg = 0, xyz[3];

                i2c.write(MMA7660_ADDR, &reg, 1, true);
                i2c.read(MMA7660_ADDR, xyz, 3);
                int x = six_bit(xyz[0]), y = six_bit(xyz[1]), z = six_bit(xyz[2]);
                int t = read_temp_x10();

                lcd.clear();
                sprintf(text, "Temp %d.%d C", t / 10, abs(t % 10));
                lcd.text(0, 0, text);
                sprintf(text, "X %3d Y %3d Z %3d", x, y, z);
                lcd.text(0, 12, text);
                lcd.text(0, 24, "tilt the board");
                lcd.update();

                printf("T=%d.%d C  X=%d Y=%d Z=%d\r\n", t / 10, abs(t % 10), x, y, z);
                ThisThread::sleep_for(500ms);
        }
}
```

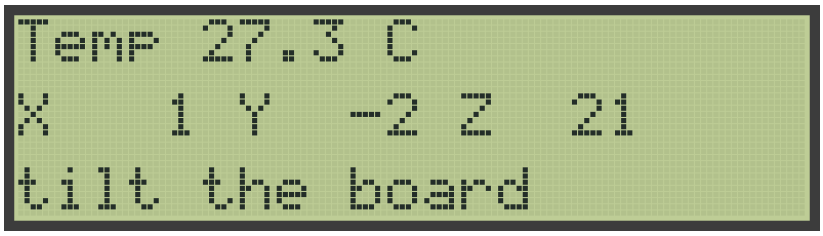


Figure - LCD of example 10 lying flat (Z about 21 = 1 g).

Serial monitor 115200 - example 10 (expected output)

```
T=27.3 C  X=1 Y=-2 Z=21
T=27.3 C  X=1 Y=-1 Z=21
T=27.4 C  X=14 Y=0 Z=16   <- board tilted
```

Serial output; X and Z change when the board is tilted.

Try this

Move a dot on the LCD with the tilt (x = 64 + X*2, y = 16 + Y).

10. Worked example: the Test 1 paper on the LPC1768

Test 1 (NUC140) asked for: a Timer countdown on the 7-segment from 77 to 5 every 0.5 s, keypad keys to change the delay of a running-LED pattern, a Timer2 bar on the left of the LCD that grows over time, and a push button adding boxes beside the bar - **all working at the same time**. Here is the same paper done on the LPC1768 with this board's parts.

Test 1 task	NUC140 part	LPC1768 version
1 countdown 77 to 5, 0.5 s	Timer1 + 7-segment	Ticker t_count 500ms, big number on the LCD (scale 3)
2 delay up / down	keypad top-middle / bottom-middle	joystick up = +50 ms, down = -50 ms; running light on LED1-LED4
3 bar on the left grows	Timer2 + LCD	Ticker t_bar 300ms, bar x 0-9 grows 1 px up, restarts when full
4 boxes beside the bar	push button SW_INT	joystick centre adds a 7x7 box (0 to 12, then back to 0)

10.1 How it is designed

- **Tickers only change numbers** (`number`, `bar_h`), both `volatile`.
- **The main loop** reads the joystick (with edge detection), moves the LEDs when the LED Timer is due, and redraws the whole LCD every 10 ms.
- Nothing in the loop waits for long, so all four tasks look simultaneous - this is what 'all tasks should work simultaneously' means in the marking.

```
//
// Example 11 : Worked example - the Test 1 paper redone for the LPC1768
//
//   Task 1 : Ticker t_count (0.5 s) counts DOWN from 77 to 5, then starts at 77
//            again; shown big on the right of the LCD
//   Task 2 : running light on LED1 -> LED4 (polling with a Timer in the main loop)
//            joystick up   = increase delay (+50 ms, slower)
//            joystick down = decrease delay (-50 ms, faster), 50 ~ 1000 ms
//   Task 3 : Ticker t_bar (0.3 s) makes the bar on the LEFT of the LCD 1 px
//            higher each time; starts again from 0 when it is full (32 px)
//   Task 4 : joystick centre adds a box beside the bar (starts with 0 boxes,
//            max 12, then back to 0)
//   All tasks run at the same time: Tickers only change variables,
//   the main loop reads the joystick, moves the LEDs and redraws the LCD.
//
//   Joystick : down p12, centre p14, up p15 (1 = pressed)
//   LCD      : MOSI p5, RESET p6, SCK p7, A0 p8, nCS p11 (lcd_c12832.h)
//   LED1 ~ LED4 on the mbed module (1 = on)
//
#include "mbed.h"
#include "lcd_c12832.h"

#define COUNT_START     77
#define COUNT_END       5
#define MAX_BOXES       12

DigitalOut leds[4] = { DigitalOut(LED1), DigitalOut(LED2),
                       DigitalOut(LED3), DigitalOut(LED4) };
DigitalIn  joy_up(p15, PullDown);
DigitalIn  joy_down(p12, PullDown);
DigitalIn  joy_centre(p14, PullDown);
LCD_C12832 lcd;
Ticker     t_count, t_bar;
Timer      t_led;

volatile int number = COUNT_START;      // task 1, changed by t_count
volatile int bar_h  = 0;                // task 3, changed by t_bar

// task 1 : Ticker callback, interrupt context -> variables only
void count_down()
{
```

```
        if (number > COUNT_END)
                number--;
        else
                number = COUNT_START;
}

// task 3 : Ticker callback
void grow_bar()
{
        bar_h = (bar_h < 32) ? bar_h + 1 : 0;
}

int main()
{
        int  delay_ms = 300;            // task 2 LED step
        int  led_pos = 0;
        int  boxes = 0;                 // task 4
        bool last_up = false, last_down = false, last_centre = false;
        char text[24];

        lcd.init();
        t_count.attach(&count_down, 500ms);
        t_bar.attach(&grow_bar, 300ms);
        t_led.start();

        while (true)
        {
                // ---- inputs (edge detection = one action per press) ----
                bool up = joy_up, down = joy_down, centre = joy_centre;

                if (up && !last_up && delay_ms < 1000)  delay_ms += 50; // increase delay
                if (down && !last_down && delay_ms > 50) delay_ms -= 50;        // decrease delay
                if (centre && !last_centre)
                        boxes = (boxes < MAX_BOXES) ? boxes + 1 : 0;
                last_up = up;
                last_down = down;
                last_centre = centre;

                // ---- task 2 : running LEDs, polling a Timer ----
                if (t_led.elapsed_time() >= std::chrono::milliseconds(delay_ms))
                {
                        t_led.reset();
                        led_pos = (led_pos + 1) % 4;
                        for (int i = 0; i < 4; i++)
                                leds[i] = (i == led_pos);
                }

                // ---- LCD : draw everything, then one update ----
                lcd.clear();

                // task 3 : bar on the left, grows upward
                lcd.box(0, 0, 10, 32);
                lcd.fill_box(1, 32 - bar_h, 8, bar_h);

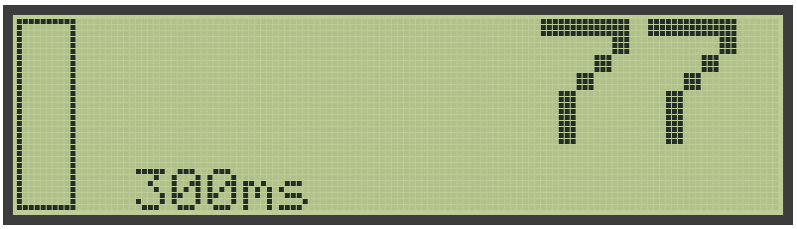
                // task 4 : boxes beside the bar, 2 rows of 6 (7 x 7 px each)
                for (int i = 0; i < boxes; i++)
                        lcd.box(14 + (i % 6) * 10, 4 + (i / 6) * 12, 7, 7);

                // task 1 : count-down number, scale 3 = 18 x 24 px per digit
                sprintf(text, "%2d", (int)number);
                lcd.text(88, 0, text, 3);

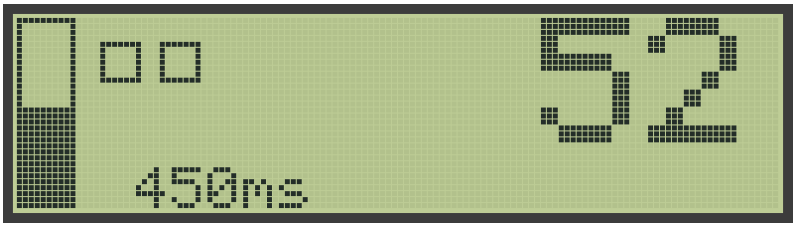
                // task 2 : the current LED delay
                sprintf(text, "%4dms", delay_ms);
                lcd.text(14, 25, text);

                lcd.update();
                ThisThread::sleep_for(10ms);
        }
}
```

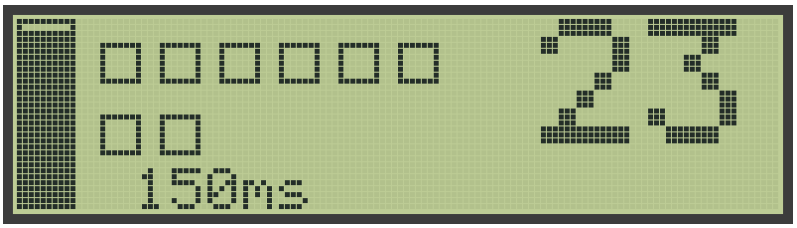
10.2 What the LCD shows



Start: 77, empty bar, no boxes, delay 300 ms.



Later: 52, bar 17 px, centre pressed twice, delay changed to 450 ms (up pressed 3 times).



Later: 23, bar almost full, 8 boxes in two rows, delay 150 ms.

10.3 Changing it for a different question

If the question says ...	Change
count UP from 5 to 77	start number = 5, Ticker does number++ and goes back to 5 after 77
every 1 s / 0.25 s	t_count.attach(&count_down, 1s) / 250ms
show on LEDs in binary	leds[i] = (number >> i) & 1;
running LEDs go the other way	led_pos = (led_pos + 3) % 4;
bar grows to the right / along the bottom	fill_box(0, 26, bar_w, 6) with bar_w 0 to 127
boxes appear under the bar	change the x/y formula in the boxes loop
use the pot to set the speed	delay_ms = 50 + pot1.read() * 950;
print to the PC as well	printf in main when number changes (never in the Ticker)

11. RTOS - Mbed OS threads

Mbed OS 6 **is** a real-time operating system (based on Keil RTX). Every program you built already runs on it: `main()` is a thread. RTOS means you can split the program into several **threads** - each one a function with its own `while (true)` loop - and the RTOS **scheduler** switches between them. When a thread sleeps or waits, the others run. Nothing to install or turn on: just `#include "mbed.h"`.

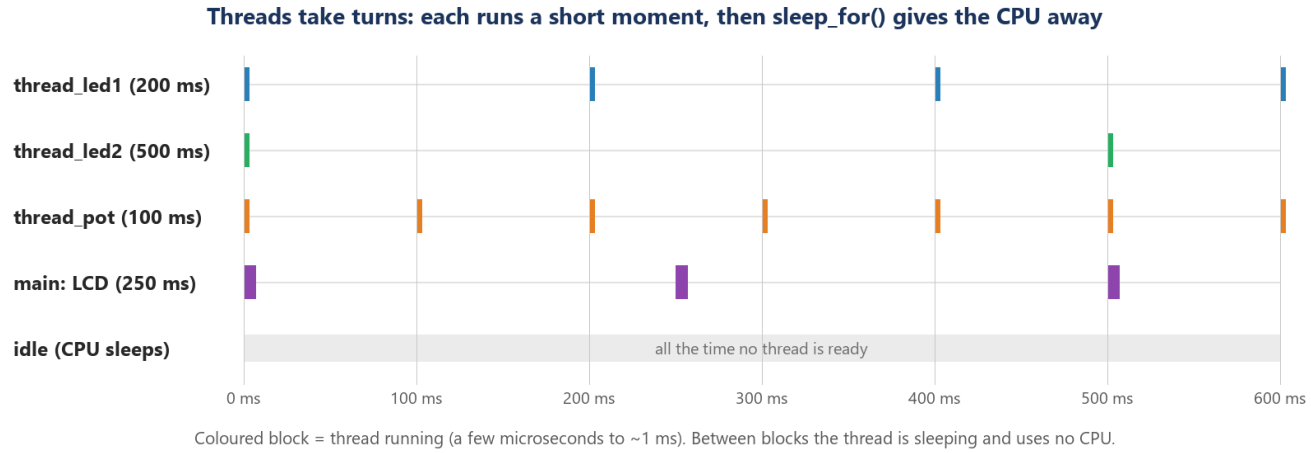


Figure - Example 12 on a timeline: four threads at four rates. Each runs for a moment, then sleeps; the CPU is idle the rest of the time.

11.1 Super loop, Ticker or threads?

	Super loop + Timer (polling)	Ticker / InterruptIn	RTOS threads
Idea	one while loop checks everything	interrupt calls a short function	one function per task, each with its own loop
Timing	depends on the whole loop	exact	good (sleep_for is accurate to ~1 ms)
printf / LCD	yes	**no**	yes (LCD needs a Mutex if 2+ threads draw)
Waiting inside a task	blocks everything	not allowed	fine - only that thread waits
Sharing data	simple variables	volatile variables	Mutex, Mail, Queue, EventFlags
Best for	small programs, tests	exact periodic ticks, buttons	many independent tasks, the project, bonus marks

11.2 Words you need (theory)

Word	Meaning
Thread (task)	an independent function with its own stack, run by the RTOS
Scheduler	the RTOS part that decides which ready thread runs next
Priority	osPriorityLow / Normal / AboveNormal / High / Realtime. The highest-priority READY thread always runs
Pre-emption	a higher-priority thread that becomes ready interrupts a lower one immediately
Round robin	threads of the same priority share the CPU in turns (time slices)
Context switch	saving one thread's registers/stack pointer and loading another's
States	**Running** (using the CPU), **Ready** (wants the CPU), **Waiting/Blocked** (sleeping, waiting for a mutex, flag, message), **Inactive** (finished / not started)
Blocking call	a call that puts the thread into Waiting: sleep_for, lock, acquire, wait_any, try_get_for
Stack	each thread's own memory for local variables and calls (default 4 KB in Mbed OS)

Word	Meaning
Race condition	two threads change the same data at the same time -> wrong values. Fix: Mutex
Deadlock	two threads each wait for a mutex the other holds -> both stop for ever
Starvation	a low-priority thread never runs because higher ones never sleep
Idle thread	runs when nothing else is ready; puts the CPU to sleep

11.3 RTOS API (Mbed OS 6)

Class / function	What it does	ISR safe
Thread t(osPriorityNormal, 2048);	create a thread object (priority, stack bytes)	-
t.start(task);	start running void task() in the new thread	no
t.join(); t.terminate();	wait for it to finish / kill it	no
ThisThread::sleep_for(100ms);	this thread waits; others run	no
ThisThread::get_id(), t.get_state()	which thread, its state	no
Mutex m; m.lock(); m.unlock();	only one thread at a time between lock and unlock	**no**
m.trylock_for(10ms)	lock if possible within 10 ms (true/false)	no
Semaphore s(0); s.acquire(); s.release();	count of tokens; acquire waits for a token, release adds one	release: yes
s.try_acquire_for(1s)	wait max 1 s (true/false)	no
EventFlags ef; ef.set(0x1);	set flag bits (wake waiting threads)	yes
ef.wait_any(0x3); ef.wait_all(0x3);	wait until any / all bits are set; returns the bits (and clears them)	no
Mail<T, N> mb; T *p = mb.try_alloc();	get an empty message slot (NULL if full)	yes
mb.put(p); p = mb.try_get_for(1s); mb.free(p);	send / receive (NULL on timeout) / give the slot back	put: yes
Queue<T, N> q; q.try_put(ptr);	send a pointer (or a small number cast to void*)	yes
q.try_get_for(1s, &ptr)	receive, true if something came	0ms: yes
EventQueue eq(32 * EVENTS_EVENT_SIZE);	a list of functions to run later in one thread	-
eq.call(f); eq.call_in(2s, f); eq.call_every(500ms, f);	run f now / once later / repeatedly	yes
t.start(callback(&eq, &EventQueue::dispatch_forever));	a thread that runs the queued functions	-

11.4 Which tool for which job

You want to ...	Use
run 2-5 independent jobs at different rates	one Thread each, loop + sleep_for (example 12)
let two threads use the LCD / the same variables	Mutex around the shared part (example 13), or only ONE thread draws (example 17)
send readings from one thread to another	Mail with a struct (example 14)
get data out of an interrupt	Queue.try_put or Mail.put from the ISR (example 14)
wake a thread when a button is pressed	EventFlags.set in the ISR, wait_any in the thread (example 15)
wake a thread every second from a Ticker / count events	Semaphore.release in the ISR, acquire in the thread (example 15)
do printf / LCD because of an interrupt	EventQueue: ISR calls queue.call(f) (example 16)
periodic job that may printf	queue.call_every(500ms, f) instead of a Ticker (example 16)

11.5 RTOS rules (read before the examples)

Rules

Every thread loop must block (sleep_for, wait_any, acquire, try_get_for ...). A thread that spins in a loop with no wait starves all lower-priority threads. Don't use the Timer busy-wait loop inside threads.

Stack size: 1024 bytes for a tiny loop, **2048** if the thread uses printf / sprintf, 4096 for an EventQueue thread. Too small = 'stack overflow' fatal error. Big arrays: make them global.

The LPC1768 has about 20 KB free for thread stacks: keep to roughly **6 threads of 2 KB**.

Interrupts still cannot printf, lock a Mutex or wait. From an ISR you may only: set EventFlags, release a Semaphore, Queue.try_put, Mail.try_alloc/put, EventQueue.call.

printf is safe from threads (Mbed locks the serial port for you); lines from different threads may come out in any order.

The LCD driver is NOT thread safe: either one thread does all the drawing, or every LCD use is inside the same Mutex.

Keep mutex locks short; never sleep while holding one; if you need two mutexes, always lock them in the same order (no deadlock).

Mbed OS 6 gotcha: `btn.rise(queue.event(f))` does not compile ('F must be TriviallyCopyable'). Use an ISR with `queue.call(f)`.

Example 12 - Threads: four tasks at four rates

- Three extra threads + main. Each thread = function with its own loop and sleep_for.
- The counters are only written by one thread each and read by main, so no mutex is needed here.

```
//
// Example 12 : RTOS Thread - three tasks running at the same time
//
//   thread_led1 : LED1 toggles every 200 ms
//   thread_led2 : LED2 toggles every 500 ms
//   thread_pot  : reads POT1 every 100 ms
//   main()      : is also a thread; it shows the counters and POT1 on the LCD
//                 every 250 ms
//
//   Each thread is a normal function with its own while (true) loop.
//   ThisThread::sleep_for() gives the CPU to the other threads (no busy waiting).
//   Stack size: 1024 bytes is enough for a small loop, use 2048 if the
//   thread calls printf / sprintf.
//
//   POT1 p19   LCD p5 p6 p7 p8 p11   LED1 LED2 on the mbed module
//
#include "mbed.h"
#include "lcd_c12832.h"

DigitalOut led1(LED1), led2(LED2);
AnalogIn   pot1(p19);
LCD_C12832 lcd;

Thread thread_led1(osPriorityNormal, 1024);     // priority, stack size (bytes)
Thread thread_led2(osPriorityNormal, 1024);
Thread thread_pot(osPriorityNormal, 1024);

volatile int n1 = 0, n2 = 0;            // how many times each LED thread ran
volatile int pot_value = 0;             // written by thread_pot, read by main

void led1_task()
{
        while (true) {
                led1 = !led1;
                n1++;
                ThisThread::sleep_for(200ms);
        }
}
```

```
void led2_task()
{
        while (true) {
                led2 = !led2;
                n2++;
                ThisThread::sleep_for(500ms);
        }
}

void pot_task()
{
        while (true) {
                pot_value = pot1.read_u16() >> 4;       // 0 ~ 4095
                ThisThread::sleep_for(100ms);
        }
}

int main()
{
        char text[24];

        lcd.init();
        thread_led1.start(led1_task);           // threads start running now
        thread_led2.start(led2_task);
        thread_pot.start(pot_task);

        while (true) {                          // main is the 4th thread
                lcd.clear();
                lcd.text(0, 0, "3 threads + main");
                sprintf(text, "LED1 %4d LED2 %4d", n1, n2);
                lcd.text(0, 11, text);
                sprintf(text, "POT1 %4d", pot_value);
                lcd.text(0, 22, text);
                lcd.update();
                ThisThread::sleep_for(250ms);
        }
}
```

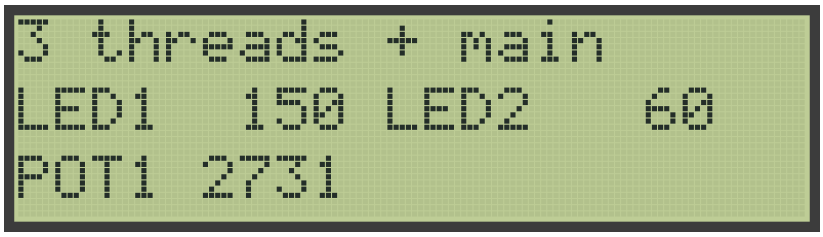


Figure - LCD of example 12 after about 30 s: the LED threads ran 150 and 60 times.

> **Try this**
>
> Give thread_pot osPriorityHigh and remove its sleep_for. What happens to the LEDs and the LCD? (They stop: starvation.) Put the sleep back.

Example 13 - Mutex: two threads share one LCD

- Both threads draw into the same buffer and use the same SPI bus. The Mutex makes each draw-and-update happen as one piece.
- Each thread erases only its own half (fill_box with false), not the whole screen.

```
//
// Example 13 : RTOS Mutex - two threads sharing the LCD safely
//
//   thread_pot draws the top half of the LCD (POT1 bar) every 100 ms
//   thread_joy draws the bottom half (joystick direction) every 50 ms
//   Both use the SAME lcd buffer and SPI bus, so only one may draw at a time:
//   lcd_mutex.lock() ... lcd_mutex.unlock() around every LCD use.
//
//   Without the mutex, one thread can clear() or update() while the other is
```

```
//   half way through drawing -> flicker, missing parts or garbage on the LCD.
//
//   Mutex rules: lock as short as possible, always unlock, never in an ISR.
//
//   POT1 p19   Joystick down p12, left p13, centre p14, up p15, right p16
//
#include "mbed.h"
#include "lcd_c12832.h"

AnalogIn   pot1(p19);
DigitalIn  joy_up(p15, PullDown), joy_down(p12, PullDown);
DigitalIn  joy_left(p13, PullDown), joy_right(p16, PullDown);
DigitalIn  joy_centre(p14, PullDown);
LCD_C12832 lcd;
Mutex      lcd_mutex;                  // protects lcd (buffer + SPI)

Thread thread_pot(osPriorityNormal, 2048);
Thread thread_joy(osPriorityNormal, 2048);

void pot_task()
{
        char text[24];

        while (true) {
                int v = pot1.read_u16() >> 4;

                lcd_mutex.lock();                       // ---- only this thread draws now
                lcd.fill_box(0, 0, 128, 16, false);     // erase the top half only
                sprintf(text, "POT1 %4d", v);
                lcd.text(0, 0, text);
                lcd.box(60, 0, 68, 7);
                lcd.fill_box(61, 1, v * 66 / 4095, 5);
                lcd.update();
                lcd_mutex.unlock();                     // ---- other threads may draw
                ThisThread::sleep_for(100ms);
        }
}

void joy_task()
{
        while (true) {
                const char *dir = "-";

                if (joy_up)     dir = "UP";
                if (joy_down)   dir = "DOWN";
                if (joy_left)   dir = "LEFT";
                if (joy_right)  dir = "RIGHT";
                if (joy_centre) dir = "CENTRE";

                lcd_mutex.lock();
                lcd.fill_box(0, 16, 128, 16, false);    // erase the bottom half only
                lcd.text(0, 20, "Joystick:");
                lcd.text(60, 20, dir);
                lcd.update();
                lcd_mutex.unlock();
                ThisThread::sleep_for(50ms);
        }
}

int main()
{
        lcd.init();                             // before the threads start
        thread_pot.start(pot_task);
        thread_joy.start(joy_task);

        while (true)                            // main has nothing else to do
                ThisThread::sleep_for(1s);
}
```

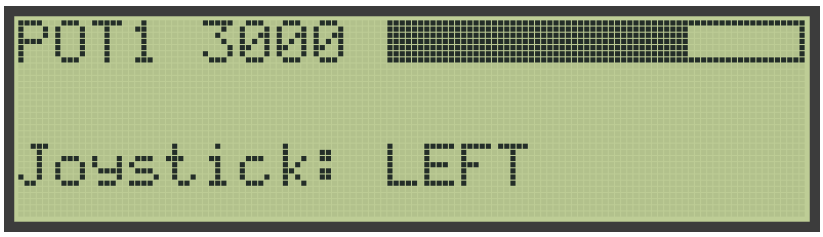


Figure - Top half drawn by thread_pot, bottom half by thread_joy (joystick held left).

> **Try this**
>
> Comment out the lock/unlock lines and run it: you will see flicker and broken pieces. Put them back.

Example 14 - Mail and Queue: passing data

- **Mail** carries a whole struct (copied into the mailbox). The receiver must call free().
- **Queue** carries pointers; here small numbers (1, 2, 3) are cast to void* so the ISR doesn't need memory.
- main waits with try_get_for(1s): it sleeps until a message arrives (no polling).

```
//
// Example 14 : RTOS Mail and Queue - sending data between threads (and from an ISR)
//
//   producer thread : reads POT1 + POT2 every 200 ms and sends them as Mail
//   joystick ISR    : puts the pressed button (1 ~ 5) into a Queue
//   consumer (main) : waits for Mail, shows the values on the LCD,
//                     prints them, and also shows the last joystick button
//
//   Mail<T, N>  : a box of N messages of type T (struct). The sender asks for an
//                 empty slot (try_alloc), fills it, put()s it; the receiver
//                 gets it (try_get_for) and must free() it.
//   Queue<T, N> : passes POINTERS (or small numbers cast to pointers).
//   try_alloc, put, try_put are ISR safe -> a good way to get data OUT of an ISR.
//
//   POT1 p19, POT2 p20   Joystick down p12, left p13, centre p14, up p15, right p16
//
#include "mbed.h"
#include "lcd_c12832.h"

typedef struct {
        int pot1;
        int pot2;
        int seq;                        // message number
} pot_msg_t;

Mail<pot_msg_t, 8>  pot_mail;           // up to 8 messages waiting
Queue<void, 8>      joy_queue;          // button numbers from the ISR

AnalogIn    pot1(p19), pot2(p20);
InterruptIn joy_up(p15), joy_down(p12), joy_centre(p14);
LCD_C12832  lcd;
Thread      producer(osPriorityNormal, 1024);

void producer_task()
{
        int seq = 0;

        while (true) {
                pot_msg_t *m = pot_mail.try_alloc();    // get an empty slot
                if (m != NULL) {                        // NULL = mailbox full
                        m->pot1 = pot1.read_u16() >> 4;
                        m->pot2 = pot2.read_u16() >> 4;
                        m->seq  = ++seq;
                        pot_mail.put(m);                // send
                }
                ThisThread::sleep_for(200ms);
        }
}

// ISR functions: only try_put a number into the queue
```

```
void on_up()     { joy_queue.try_put((void *)1); }
void on_down()   { joy_queue.try_put((void *)2); }
void on_centre() { joy_queue.try_put((void *)3); }

int main()
{
        char text[24];
        int  last_button = 0;

        lcd.init();
        joy_up.mode(PullDown);     joy_up.rise(&on_up);
        joy_down.mode(PullDown);   joy_down.rise(&on_down);
        joy_centre.mode(PullDown); joy_centre.rise(&on_centre);
        producer.start(producer_task);

        while (true) {
                // wait up to 1 s for the next pot message
                pot_msg_t *m = pot_mail.try_get_for(1s);

                // empty the button queue without waiting (0ms)
                void *b;
                while (joy_queue.try_get_for(0ms, &b))
                        last_button = (int)b;

                lcd.clear();
                if (m != NULL) {
                        sprintf(text, "#%d", m->seq);
                        lcd.text(0, 0, text);
                        sprintf(text, "POT1 %4d POT2 %4d", m->pot1, m->pot2);
                        lcd.text(0, 11, text);
                        printf("#%d POT1 %d POT2 %d\r\n", m->seq, m->pot1, m->pot2);
                        pot_mail.free(m);               // ALWAYS give the slot back
                } else {
                        lcd.text(0, 11, "no mail for 1 s");
                }
                sprintf(text, "button %d (1U 2D 3C)", last_button);
                lcd.text(0, 22, text);
                lcd.update();
        }
}
```

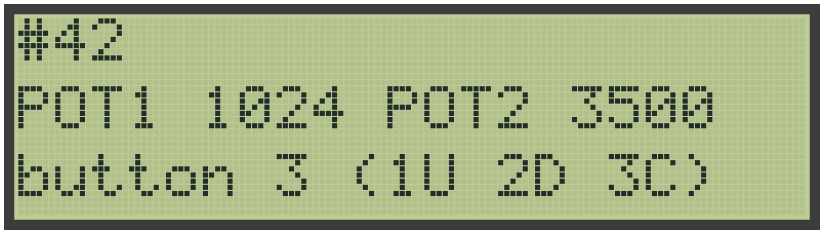


Figure - Message #42 from the producer thread, and the last joystick button (3 = centre) from the ISR queue.

> **Try this**
>
> Add a third field 'pot_percent' to the struct and show it. Make the producer send only when a pot changes by more than 50.

Example 15 - EventFlags and Semaphore: waking threads from ISRs

- The ISRs are one line each. All slow work (printf, LCD) is in the threads.
- EventFlags = several yes/no events in one 32-bit word; wait_any returns which ones happened.
- Semaphore = a counter of tokens: the Ticker adds one per second, the thread takes one each time round.

```
//
// Example 15 : RTOS EventFlags and Semaphore - waking threads from interrupts
//
//   EventFlags : the joystick ISRs set a flag bit; thread_flags waits for any
//                bit and then does the slow work (printf, LCD).
//                UP = bit 0, DOWN = bit 1, CENTRE = bit 2
//   Semaphore  : a Ticker releases the semaphore every 1 s; thread_sem waits
```

```
//               (acquire) and toggles LED2 + counts seconds.
//   The threads sleep while waiting, so they use no CPU.
//
//   EventFlags.set() and Semaphore.release() are ISR safe.
//   wait_any() / acquire() are NOT: call them only from threads.
//
//   Joystick down p12, centre p14, up p15   LED1 LED2 on the mbed module
//
#include "mbed.h"
#include "lcd_c12832.h"

#define FLAG_UP         (1UL << 0)
#define FLAG_DOWN       (1UL << 1)
#define FLAG_CENTRE     (1UL << 2)

EventFlags  joy_flags;
Semaphore   second_sem(0);              // starts with 0 tokens -> acquire() waits
Ticker      tick;
InterruptIn joy_up(p15), joy_down(p12), joy_centre(p14);
DigitalOut  led1(LED1), led2(LED2);
LCD_C12832  lcd;
Mutex       lcd_mutex;

Thread thread_flags(osPriorityNormal, 2048);
Thread thread_sem(osPriorityNormal, 2048);

volatile int value = 50;                // changed by UP / DOWN
volatile int seconds = 0;

// ---- ISRs: just set a flag / release a token ----
void on_up()     { joy_flags.set(FLAG_UP); }
void on_down()   { joy_flags.set(FLAG_DOWN); }
void on_centre() { joy_flags.set(FLAG_CENTRE); }
void on_tick()   { second_sem.release(); }

void show()                             // draw everything (called from both threads)
{
        char text[24];

        lcd_mutex.lock();
        lcd.clear();
        sprintf(text, "value %3d", value);
        lcd.text(0, 0, text);
        lcd.fill_box(0, 11, value * 128 / 100, 6);
        sprintf(text, "time %3d s", seconds);
        lcd.text(0, 22, text);
        lcd.update();
        lcd_mutex.unlock();
}

void flags_task()
{
        while (true) {
                // sleep until UP, DOWN or CENTRE happens; the bits are cleared on return
                uint32_t f = joy_flags.wait_any(FLAG_UP | FLAG_DOWN | FLAG_CENTRE);

                if ((f & FLAG_UP) && value < 100)  value += 10;
                if ((f & FLAG_DOWN) && value > 0)  value -= 10;
                if (f & FLAG_CENTRE)               value = 50;
                led1 = !led1;
                printf("flags 0x%lX -> value %d\r\n", (unsigned long)f, value);
                show();
                ThisThread::sleep_for(150ms);   // simple debounce
                joy_flags.clear();              // drop presses from switch bounce
        }
}

void sem_task()
{
        while (true) {
                second_sem.acquire();           // sleep until the Ticker releases
                seconds++;
                led2 = !led2;
                show();
```

```
        }
}

int main()
{
        lcd.init();
        joy_up.mode(PullDown);     joy_up.rise(&on_up);
        joy_down.mode(PullDown);   joy_down.rise(&on_down);
        joy_centre.mode(PullDown); joy_centre.rise(&on_centre);
        thread_flags.start(flags_task);
        thread_sem.start(sem_task);
        tick.attach(&on_tick, 1s);
        show();

        while (true)
                ThisThread::sleep_for(1s);
}
```

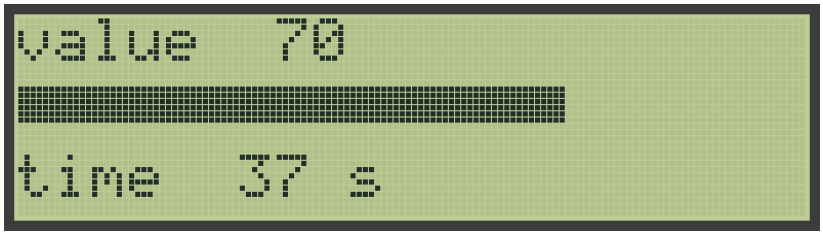


Figure - value changed to 70 with joystick up (2 presses), 37 seconds counted by the semaphore thread.

> **Try this**
>
> Add FLAG_LEFT / FLAG_RIGHT (bits 3 and 4) that move a small box left/right.

Example 16 - EventQueue: interrupts that may printf

- Everything in the queue runs in one thread, one function at a time, so the LCD needs no mutex.
- call_every replaces a Ticker when the periodic job needs printf / LCD. call_in replaces a Timeout.

```
//
// Example 16 : RTOS EventQueue - the easy way to do slow work "from" an interrupt
//
//   An EventQueue runs on its own thread (dispatch_forever). Anything you post
//   to it runs later in THAT thread, so printf and the LCD are allowed there.
//
//   queue.call_every(500ms, f) : f every 500 ms (like a Ticker, but in a thread)
//   queue.call(f)              : run f once, as soon as possible (ISR safe)
//   ISR { queue.call(f); }     : the button interrupt posts f to the queue
//                                -> f may use printf / LCD
//   (btn.rise(queue.event(f)) does NOT compile in Mbed OS 6 unless the option
//    platform.callback-nontrivial is turned on, so use a 1-line ISR instead)
//   queue.call_in(2s, f)       : run f once, 2 s from now (like a Timeout)
//
//   Joystick up p15 = +1, down p12 = -1, centre p14 = message for 2 s
//   POT1 p19 shown every 500 ms
//
#include "mbed.h"
#include "lcd_c12832.h"

EventQueue  queue(32 * EVENTS_EVENT_SIZE);      // room for 32 events
Thread      event_thread(osPriorityNormal, 4096);
InterruptIn joy_up(p15), joy_down(p12), joy_centre(p14);
AnalogIn    pot1(p19);
DigitalOut  led1(LED1);
LCD_C12832  lcd;

int  number = 0;
bool show_msg = false;

// everything below runs in event_thread, one at a time -> no mutex needed
void redraw()
{
```

```
        char text[24];

        lcd.clear();
        sprintf(text, "number %d", number);
        lcd.text(0, 0, text);
        sprintf(text, "POT1 %4d", pot1.read_u16() >> 4);
        lcd.text(0, 11, text);
        if (show_msg)
                lcd.text(0, 22, "centre pressed!");
        lcd.update();
}

void up()       { number++; printf("up   -> %d\r\n", number); redraw(); }
void down()     { number--; printf("down -> %d\r\n", number); redraw(); }
void msg_off()  { show_msg = false; redraw(); }
void centre()
{
        show_msg = true;
        printf("centre\r\n");
        redraw();
        queue.call_in(2s, msg_off);             // hide the message after 2 s
}
void blink()    { led1 = !led1; redraw(); }

// ISRs: only post the real work to the queue (queue.call is ISR safe)
void on_up()     { queue.call(up); }
void on_down()   { queue.call(down); }
void on_centre() { queue.call(centre); }

int main()
{
        lcd.init();
        joy_up.mode(PullDown);     joy_up.rise(&on_up);
        joy_down.mode(PullDown);   joy_down.rise(&on_down);
        joy_centre.mode(PullDown); joy_centre.rise(&on_centre);
        queue.call_every(500ms, blink);

        // the queue's thread runs the events forever
        event_thread.start(callback(&queue, &EventQueue::dispatch_forever));

        while (true)
                ThisThread::sleep_for(1s);
}
```

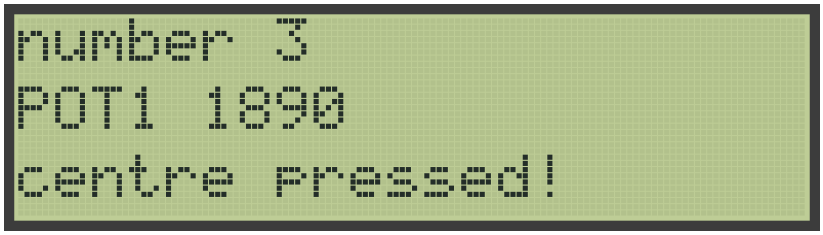


Figure - number = 3 after three 'up' presses, centre pressed less than 2 s ago.

> **Try this**
>
> Use call_every(100ms, ...) to read POT1 and move a bar; keep the buttons working.

Example 17 - The Test 1 paper with RTOS threads

Same behaviour and LCD layout as example 11 (section 10), but each task is its own thread. Compare the two:

	Example 11 (Tickers + super loop)	Example 17 (RTOS threads)
Countdown	Ticker ISR changes a volatile int	t_count thread: sleep_for(500ms), then change
Running LEDs	Timer polled in the main loop	t_leds thread sleeps for led_delay
Joystick	edge detection in the main loop	t_input thread (higher priority) every 20 ms
Bar	Ticker ISR	t_bar thread

	Example 11 (Tickers + super loop)	Example 17 (RTOS threads)
LCD	main loop every 10 ms	main thread every 50 ms (only thread that draws)
Shared data	volatile	data_mutex; main copies all values at once
Good for	short, fewer rules	each task easy to read and change alone; shows RTOS knowledge

```
//
// Example 17 : Worked example 11 (Test 1 paper) rewritten with RTOS threads
//
//   One thread per task, each with its own timing:
//     t_count : count down 77 -> 5 every 500 ms, then 77 again   (task 1)
//     t_leds  : running light LED1 -> LED4, delay = led_delay    (task 2)
//     t_input : reads the joystick every 20 ms                   (tasks 2 + 4)
//               up = +50 ms delay, down = -50 ms, centre = add a box
//     t_bar   : bar on the left grows 1 px every 300 ms          (task 3)
//     main    : redraws the LCD every 50 ms (the only thread using the LCD)
//   Shared numbers are protected by data_mutex; only main draws, so the LCD
//   needs no mutex. Same LCD layout as example 11.
//
//   Joystick down p12, centre p14, up p15   LCD p5 p6 p7 p8 p11   LED1 ~ LED4
//
#include "mbed.h"
#include "lcd_c12832.h"

#define COUNT_START     77
#define COUNT_END       5
#define MAX_BOXES       12

DigitalOut leds[4] = { DigitalOut(LED1), DigitalOut(LED2),
                       DigitalOut(LED3), DigitalOut(LED4) };
DigitalIn  joy_up(p15, PullDown), joy_down(p12, PullDown), joy_centre(p14, PullDown);
LCD_C12832 lcd;
Mutex      data_mutex;          // protects the 4 shared values below

int number    = COUNT_START;    // task 1
int led_delay = 300;            // task 2 (ms)
int bar_h     = 0;              // task 3
int boxes     = 0;              // task 4

Thread t_count(osPriorityNormal, 1024);
Thread t_leds(osPriorityNormal, 1024);
Thread t_input(osPriorityAboveNormal, 1024);    // inputs react first
Thread t_bar(osPriorityNormal, 1024);

void count_task()
{
        while (true) {
                ThisThread::sleep_for(500ms);
                data_mutex.lock();
                number = (number > COUNT_END) ? number - 1 : COUNT_START;
                data_mutex.unlock();
        }
}

void leds_task()
{
        int pos = 0;

        while (true) {
                for (int i = 0; i < 4; i++)
                        leds[i] = (i == pos);
                pos = (pos + 1) % 4;

                data_mutex.lock();
                int d = led_delay;              // copy, then unlock before sleeping
                data_mutex.unlock();
                ThisThread::sleep_for(std::chrono::milliseconds(d));
        }
}
```

```
void input_task()
{
        bool last_up = false, last_down = false, last_centre = false;

        while (true) {
                bool up = joy_up, down = joy_down, centre = joy_centre;

                data_mutex.lock();
                if (up && !last_up && led_delay < 1000)  led_delay += 50;
                if (down && !last_down && led_delay > 50) led_delay -= 50;
                if (centre && !last_centre)
                        boxes = (boxes < MAX_BOXES) ? boxes + 1 : 0;
                data_mutex.unlock();

                last_up = up; last_down = down; last_centre = centre;
                ThisThread::sleep_for(20ms);
        }
}

void bar_task()
{
        while (true) {
                ThisThread::sleep_for(300ms);
                data_mutex.lock();
                bar_h = (bar_h < 32) ? bar_h + 1 : 0;
                data_mutex.unlock();
        }
}

int main()
{
        char text[24];

        lcd.init();
        t_count.start(count_task);
        t_leds.start(leds_task);
        t_input.start(input_task);
        t_bar.start(bar_task);

        while (true) {
                data_mutex.lock();              // take a copy of everything at once
                int n = number, d = led_delay, h = bar_h, b = boxes;
                data_mutex.unlock();

                lcd.clear();
                lcd.box(0, 0, 10, 32);                          // task 3 bar
                lcd.fill_box(1, 32 - h, 8, h);
                for (int i = 0; i < b; i++)                     // task 4 boxes
                        lcd.box(14 + (i % 6) * 10, 4 + (i / 6) * 12, 7, 7);
                sprintf(text, "%2d", n);                        // task 1 number
                lcd.text(88, 0, text, 3);
                sprintf(text, "%4dms", d);                      // task 2 delay
                lcd.text(14, 25, text);
                lcd.update();
                ThisThread::sleep_for(50ms);
        }
}
```

The LCD looks exactly like example 11 (section 10.2).

11.6 RTOS questions for short answers

Question	Answer
What is an RTOS?	An OS that runs several threads and guarantees time limits by priority-based pre-emptive scheduling.
Thread vs ISR?	A thread can block, wait and use printf/mutex; an ISR must be short and can't block. ISRs always beat threads.
Why a Mutex?	To stop a race condition: only one thread at a time uses shared data or a device (e.g. the LCD).

Question	Answer
Mutex vs Semaphore?	Mutex = one owner, lock/unlock by the same thread, for protection. Semaphore = counter of tokens, any thread/ISR can release, for signalling or limiting.
What is a deadlock? How to avoid it?	Threads waiting for each other's locks for ever. Lock in a fixed order, hold locks briefly, use trylock_for.
What is starvation?	A low-priority thread never runs because higher ones never block.
Why must threads sleep or wait?	So lower-priority and equal threads get the CPU, and the idle thread can save power.
How to get data from an ISR to a thread?	Queue.try_put / Mail.put, EventFlags.set, Semaphore.release or EventQueue.call.
Thread states?	Running, Ready, Waiting (Blocked), Inactive.

12. Practice question - try it yourself

Do this one without looking at a solution (there is none in this guide on purpose). Give yourself 60 minutes like a real test. Use sections 6-9 and the examples as your reference.

Task	Input	Output
1	Ticker every 1 s	LCD shows a stopwatch **MM:SS** (scale 2) that counts up from 00:00
2	Joystick centre	pause / resume the stopwatch; each pause prints **Paused at MM:SS** on the serial monitor
3	POT1	speed of a running light on LED1 - LED4, from 100 ms (POT1 max) to 1000 ms (POT1 min); show the speed in ms on the LCD
4	Joystick left / right	move a small 6x6 box along the bottom row of the LCD; it must stop at the edges
5	Joystick up	reset the stopwatch to 00:00 and the RGB LED flashes green for 0.5 s (Timeout)

All tasks must work at the same time.

Hints

- Keep seconds in a `volatile int` changed by the Ticker; minutes = s / 60, seconds = s % 60, `sprintf(t, "%02d:%02d", ...)`.
- Pause = a `volatile bool running`; the Ticker only counts when it is true (or detach/attach).
- printf for task 2 goes in main, after edge detection of the centre press - not in an interrupt.
- Task 3 is the same as example 11's LED code with `delay_ms` computed from the pot every loop.
- Task 5: `green = 0;` then `to.attach(&green_off, 500ms);` (example 04).

Self-check before you call it done

- Builds with 0 errors; header comment lists what it does and the pins.
- Stopwatch keeps exact time while you move the box and turn the pot (Ticker, not a sleep in the loop).
- One centre press = exactly one pause/resume (edge detection or debounce).
- The box cannot leave the screen; LEDs never stop running.
- Serial shows the pause line once per pause; nothing printed from interrupts.

13. Troubleshooting

Problem	Likely cause	Fix
No MBED drive appears	cable is charge-only / loose; board not powered	other USB cable or port; check the mbed's blue power LED
Copied .bin but nothing changed	reset not pressed; wrong program built; old .bin	press reset; check the .bin time stamp; check Active program
LEDs flash in a strange pattern / board stops	Mbed OS fatal error: printf / LCD / BusOut / mutex inside a Ticker or InterruptIn	move that code to main, use a volatile flag
LCD blank	lcd.init() missing, update() missing, or p5-p8/p11 used for something else	init once, update after drawing, free the LCD pins
LCD shows old pieces of text	no lcd.clear() before drawing	clear, draw, update every loop
Pot always 0 or fixed	module not seated well in the app board	re-seat the LPC1768 module
Joystick press counts many times	contact bounce / holding the key	edge detection or 200 ms debounce
Joystick never reads 1	missing PullDown, wrong pin	DigitalIn x(p14, PullDown); check the pin table
RGB LED is the wrong way round	active low	0 = on: write 1.0f - brightness
Serial monitor shows rubbish	wrong baud	115200 (or what you set in BufferedSerial)
Serial shows nothing	wrong COM port; printf of %f; program crashed	Device Manager > Ports; print integers
sleep_for seems very slow	old interface firmware on one board (woke once per second)	use a Timer busy-wait loop instead
Mbed Studio Run hangs 'waiting for debug probe'	pyOCD problem on this laptop	ignore Run; copy the .bin by hand
Build very slow every time	profile changed, or Clean build	keep the same profile; only main.cpp should recompile
Red error in editor but build OK	editor checker (FPU message)	trust the Output panel only
RTOS: fatal error 'stack overflow' / board stops after a thread starts	thread stack too small (printf, sprintf, big local arrays)	stack 2048 or 4096; make big arrays global
RTOS: one thread never runs	a higher-priority thread loops without sleeping or waiting	every thread loop must sleep_for / wait / acquire
RTOS: program freezes after a while	deadlock (two mutexes locked in a different order) or a mutex never unlocked	lock one mutex at a time; unlock on every path
RTOS: LCD flickers / garbage	two threads draw on the LCD without a mutex	one LCD thread, or lock a Mutex around every LCD use
RTOS: 'F must be TriviallyCopyable' compile error	btn.rise(queue.event(f))	ISR that calls queue.call(f) (example 16)

14. Test-day checklist

1. Plug in the board. Check the MBED drive (MBED.HTM) appears.
2. Flash the board check: copy **15 Test 2\Test2_board_check.bin** to the drive, press reset. LCD shows 'Test 2 ready', pot values, joystick letters; LED1 blinks. Now you know the board, LCD and flashing all work.
3. Open Mbed Studio, Active program = **Test2**, Target = **mbed LPC1768**.
4. Read the whole question. List each task: input, output, timing. Decide: Ticker (exact timing) or Timer (polling) for each.
5. If using Claude Code / Codex, give it the full question plus: *'LPC1768 + mbed Application Board, Mbed OS 6, program C:\Users\TUF FA707RC-HX024W\Mbed Programs\Test2, replace main.cpp, use lcd_c12832.h, no printf/LCD in Ticker callbacks, build with build_test2.cmd*'.
6. Build. Fix the first error first. Flash. Press reset.
7. Test every task **on the board**, and test them **together** (the marking checks they run simultaneously).
8. Keep the header comment (what it does + pins).
9. After the test: copy main.cpp and Test2.bin into 15 Test 2.

Things that cost marks last time

- Key / direction mapping: check on the real board which joystick direction is which before trusting the picture.
- Direction words in the paper (clockwise / counter-clockwise, increasing / decreasing): read twice, ask if unsure.
- Tasks that work alone but freeze when combined: usually a sleep or a long loop blocking the others.

Quick theory self-test

Question	Answer
Timer vs Ticker?	Timer = polling, main reads the time. Ticker = interrupt calls the function at a fixed rate; main is free.
Why no wait() in a scheduled program?	A blocking delay stops every other task, so their timing overruns and goes out of sync.
What happens if a 1 ms task takes 2 ms?	Timing overruns, the system goes out of sync / misses deadlines.
POT raw 2048 (12-bit), voltage?	2048 x 3.3 / 4095 = 1.65 V
read_u16() = 49152, 12-bit value?	49152 >> 4 = 3072 (75 %)
Speaker 1 kHz: period?	1 / 1000 = 1 ms, duty 0.5
RGB red half brightness?	red = 0.5f (active low: 1 - 0.5)
Two mbeds over UART: wiring?	TX to RX, RX to TX, GND to GND, same baud
How many UARTs on the LPC1768?	4: three on pins, one to the USB link (USBTX/USBRX)
Which mbed pins are analog in?	p15 - p20 (6 channels, 12-bit)

15. Code modules (copy and paste)

Ready pieces of code. Each one is a few helper functions, like `set_rgb()` and `set_led()`. Paste the ones you need, then call them. All 16 were compiled one by one **and all together in one program** (0 errors, 0 warnings), so you can mix them.

> **How to paste a module**
>
> Every module has up to three parts. Put each part in its place:
>
> **1. ABOVE main()** - objects, variables, functions.
>
> **2. SETUP** - inside main(), before `while (true)` (runs once).
>
> **3. LOOP** - inside `while (true)` (the LOOP part is only an example of how to call it: change it).
>
> If a module says **Needs**, paste that module too. For easy copying use **Code_Modules.txt** (keeps the tabs) instead of the PDF.

```
#include "mbed.h"
#include "lcd_c12832.h" // only if you use the LCD (M04, M05)

// 1. ABOVE main(): paste the module tops here

int main()
{
        // 2. SETUP: paste the module setup lines here

        while (true)
        {
                // 3. LOOP: your code, calling the module functions

                ThisThread::sleep_for(20ms);
        }
}
```

Module	What it gives you	Needs
M01	LED1 - LED4 helpers	-
M02	RGB LED helper	-
M03	Joystick: held and pressed-once	-
M04	LCD: text and numbers	-
M05	LCD: big number, bars, boxes	M04
M06	Ticker: count every period	-
M07	Timer: do a task every N ms (changeable speed)	-
M08	Running LEDs (forward / backward / bounce)	M01
M09	Potentiometers (ADC)	-
M10	Serial: print to the PC every 0.5 s	-
M11	Serial: read keys typed on the PC	-
M12	PWM: RGB brightness and speaker beep	-
M13	Button interrupt + debounce + Timeout	-
M14	UART between two boards (text lines)	-
M15	RTOS: a thread and a mutex	-
M16	RTOS: EventQueue (interrupt -> printf / LCD)	-

M01 - LED1 - LED4 helpers

Turn the 4 blue LEDs on/off, one at a time, as a number in binary, or as a level.

1. Paste ABOVE main()

```
// LED1 ~ LED4 on the mbed module (1 = on)
DigitalOut led[4] = { DigitalOut(LED1), DigitalOut(LED2), DigitalOut(LED3), DigitalOut(LED4) };

// each LED on (1) or off (0)
void set_led(int a, int b, int c, int d)
{
        led[0] = a;
        led[1] = b;
        led[2] = c;
        led[3] = d;
}

// only LED number n on (0 = LED1 ... 3 = LED4), the rest off
void led_one(int n)
{
        for (int i = 0; i < 4; i++)
                led[i] = (i == n);
}

// first n LEDs on (0 ~ 4), like a level meter
void led_level(int n)
{
        for (int i = 0; i < 4; i++)
                led[i] = (i < n);
}

// number 0 ~ 15 in binary (LED1 = bit 0)
void led_binary(int value)
{
        for (int i = 0; i < 4; i++)
                led[i] = (value >> i) & 1;
}
```

3. Use it in the LOOP (example)

```
set_led(1, 0, 0, 1);            // LED1 + LED4 on
led_one(2);                     // only LED3
led_level(3);                   // LED1 ~ LED3
led_binary(5);                  // 0101 -> LED1 + LED3
```

> **Adjust**
>
> LED1 ~ LED4 are 1 = on.
>
> set_led(1,0,0,1) = LED1 and LED4 on.

M02 - RGB LED helper

Set the RGB LED colour with 1 = on (the function handles active low).

1. Paste ABOVE main()

```
// RGB LED on the app board: R p23, G p24, B p25 (0 = on), start off
DigitalOut rgb[3] = { DigitalOut(p23, 1), DigitalOut(p24, 1), DigitalOut(p25, 1) };

// 1 = colour on, 0 = off
void set_rgb(int r, int g, int b)
{
        rgb[0] = !r;
        rgb[1] = !g;
        rgb[2] = !b;
}
```

3. Use it in the LOOP (example)

```
set_rgb(1, 1, 0);               // yellow
```

> **Adjust**
>
> Colours: (1,0,0) red, (0,1,0) green, (0,0,1) blue, (1,1,0) yellow, (1,0,1) purple, (0,1,1) cyan, (1,1,1) white, (0,0,0) off.

M03 - Joystick: held and pressed-once

Read all 5 directions. 'held' = true while you hold it. 'pressed' = true only once per press (edge detection).

1. Paste ABOVE main()

```
// joystick (1 = pressed)
DigitalIn joy_up(p15, PullDown), joy_down(p12, PullDown), joy_left(p13, PullDown);
DigitalIn joy_right(p16, PullDown), joy_centre(p14, PullDown);

// true for ONE loop when the button goes from released to pressed
bool up_pressed, down_pressed, left_pressed, right_pressed, centre_pressed;
bool last_up, last_down, last_left, last_right, last_centre;

void read_joystick()
{
        bool up = joy_up, down = joy_down, left = joy_left, right = joy_right, centre = joy_centre;

        up_pressed     = up && !last_up;
        down_pressed   = down && !last_down;
        left_pressed   = left && !last_left;
        right_pressed  = right && !last_right;
        centre_pressed = centre && !last_centre;

        last_up = up;  last_down = down;  last_left = left;  last_right = right;  last_centre = centre;
}
```

3. Use it in the LOOP (example)

```
read_joystick();
if (joy_up) {                   // while UP is held
}
if (centre_pressed) {           // once per CENTRE press
}
```

> **Adjust**
>
> Use joy_up etc. for 'while held'. Use up_pressed etc. for 'each press does one thing'.
>
> Call read_joystick() once at the top of every loop.

M04 - LCD: text and numbers

Start the LCD and write 3 lines of text, including numbers.

1. Paste ABOVE main()

```
#include "lcd_c12832.h"         // put this line at the very top, under #include "mbed.h"

LCD_C12832 lcd;                 // 128 x 32 pixels, (0,0) = top left
char lcd_text[24];              // for sprintf
```

2. Paste in SETUP (before while)

```
lcd.init();                     // once
```

3. Use it in the LOOP (example)

```
int value = 123;                // (your number)
lcd.clear();
sprintf(lcd_text, "Value %d", value);
```

```
lcd.text(0, 0, lcd_text);       // line 1
lcd.text(0, 11, "line 2");      // line 2
lcd.text(0, 22, "line 3");      // line 3
lcd.update();
```

Adjust

21 characters per line. Lines at y = 0, 11, 22.

Always: clear, draw everything, update once.

Numbers: sprintf(lcd_text, "...%d...", value).

M05 - LCD: big number, bars, boxes

Big number (scale 2 or 3), a bar that grows up, a bar that grows right, and a row of boxes. **Needs:** M04.

1. Paste ABOVE main()

```
// big number at (x, y), scale 2 or 3
void lcd_big_number(int x, int y, int value, int scale)
{
        char t[12];
        sprintf(t, "%d", value);
        lcd.text(x, y, t, scale);
}

// bar at x, w pixels wide, grows UP, h = 0 ~ 32
void lcd_vbar(int x, int w, int h)
{
        lcd.box(x, 0, w, 32);                   // frame
        lcd.fill_box(x + 1, 32 - h, w - 2, h);  // fill from the bottom
}

// bar at row y, 6 px high, full width, grows RIGHT, value 0 ~ max
void lcd_hbar(int y, int value, int max)
{
        lcd.box(0, y, 128, 6);
        lcd.fill_box(1, y + 1, value * 126 / max, 4);
}

// 'count' boxes of 7 x 7 px starting at (x, y), 'per_row' in a row
void lcd_boxes(int x, int y, int count, int per_row)
{
        for (int i = 0; i < count; i++)
                lcd.box(x + (i % per_row) * 10, y + (i / per_row) * 12, 7, 7);
}
```

3. Use it in the LOOP (example)

```
lcd.clear();
lcd_vbar(0, 10, 20);            // left bar, 20 px high
lcd_boxes(14, 4, 3, 6);         // 3 boxes beside it
lcd_big_number(88, 0, 77, 3);   // big 77 on the right
lcd.update();
```

Adjust

Big number: scale 2 = 12 x 16 px per digit, scale 3 = 18 x 24 px.

lcd_vbar: h = 0 ~ 32. lcd_hbar: value / max sets the length.

lcd_boxes: per_row boxes in a row, then a new row.

M06 - Ticker: count every period

A Ticker changes a number in the background (count down 77 to 5 and repeat). Includes changing the speed.

1. Paste ABOVE main()

```
Ticker tick;
volatile int counter = 77;              // read it in main, e.g. show on the LCD

// runs every period in an interrupt: only change variables here
void tick_count()
{
        if (counter > 5)
                counter--;
        else
                counter = 77;
}

// change the Ticker period
void set_tick_speed(int ms)
{
        tick.detach();
        tick.attach(&tick_count, std::chrono::milliseconds(ms));
}
```

2. Paste in SETUP (before while)

```
tick.attach(&tick_count, 500ms);        // every 0.5 s
```

3. Use it in the LOOP (example)

```
int now = counter;                      // use 'now' for the LCD / LEDs
```

> **Adjust**
>
> Change 500ms, the start/end numbers, or count up instead (counter++).
>
> Only change variables in tick_count(): no printf / LCD there.
>
> set_tick_speed(ms) changes the period (detach + attach).

M07 - Timer: do a task every N ms (changeable speed)

Polling timer for a task at a speed you can change while running, e.g. running LEDs whose delay the joystick changes.

1. Paste ABOVE main()

```
Timer task_timer;
int task_ms = 300;                      // task period in ms
```

2. Paste in SETUP (before while)

```
task_timer.start();
```

3. Use it in the LOOP (example)

```
if (task_timer.elapsed_time() >= std::chrono::milliseconds(task_ms))
{
        task_timer.reset();
        // ---- your task here (e.g. next running LED) ----
}
```

> **Adjust**
>
> task_ms is the speed: change it anywhere (joystick, pot ...).
>
> Put your task where it says.

M08 - Running LEDs (forward / backward / bounce)

One step of a running light each time you call it. Call it from a Timer (M07) or a Ticker flag. **Needs:** M01.

1. Paste ABOVE main()

```
int run_pos = 0, run_dir = 1;

void run_forward()  { run_pos = (run_pos + 1) % 4;  led_one(run_pos); }
void run_backward() { run_pos = (run_pos + 3) % 4;  led_one(run_pos); }
void run_bounce()
{
        run_pos += run_dir;
        if (run_pos >= 3 || run_pos <= 0)
                run_dir = -run_dir;
        led_one(run_pos);
}
```

3. Use it in the LOOP (example)

```
run_forward();
ThisThread::sleep_for(200ms);
```

> **Adjust**
>
> run_forward: LED1 -> LED4 -> LED1 ...
>
> run_backward: LED4 -> LED1 ...
>
> run_bounce: LED1 -> LED4 -> LED1 back and forth.

M09 - Potentiometers (ADC)

Read POT1 / POT2 as 0 ~ 4095, and convert to %, millivolts, or any range (e.g. a speed).

1. Paste ABOVE main()

```
AnalogIn pot1(p19), pot2(p20);

// average of 8 readings, 0 ~ 4095
int read_pot(AnalogIn &pot)
{
        int sum = 0;
        for (int i = 0; i < 8; i++)
                sum += pot.read_u16() >> 4;
        return sum / 8;
}

int pot_percent(int raw) { return raw * 100 / 4095; }           // 0 ~ 100
int pot_mv(int raw)      { return raw * 3300 / 4095; }          // 0 ~ 3300 mV
int map_pot(int raw, int lo, int hi) { return lo + (hi - lo) * raw / 4095; }    // lo ~ hi
```

3. Use it in the LOOP (example)

```
int p1 = read_pot(pot1);
int speed_ms = map_pot(p1, 100, 1000);
```

> **Adjust**
>
> map_pot(raw, 100, 1000) = 100 when the pot is at 0, 1000 at the top.
>
> Want it the other way round? map_pot(raw, 1000, 100).

M10 - Serial: print to the PC every 0.5 s

printf to the serial monitor (115200 from mbed_app.json) at a fixed rate, with volts printed as integers.

1. Paste ABOVE main()

```
Timer print_timer;
```

2. Paste in SETUP (before while)

```
print_timer.start();
printf("\r\nProgram start\r\n");
```

3. Use it in the LOOP (example)

```
if (print_timer.elapsed_time() >= 500ms)
{
        print_timer.reset();
        int mv = 1650;                                  // (your value)
        printf("value = %d.%02d V\r\n", mv / 1000, mv % 1000 / 10);
}
```

> **Adjust**
>
> End lines with \r\n. No %f: print mv / 1000 and mv % 1000.
>
> Never printf inside a Ticker / InterruptIn function.

M11 - Serial: read keys typed on the PC

BufferedSerial on USB: read one character without waiting, and send text back.

1. Paste ABOVE main()

```
BufferedSerial pc(USBTX, USBRX, 115200);

void pc_print(const char *s) { pc.write(s, strlen(s)); }

// one character typed on the PC, or -1 if none (does not wait)
int pc_getc()
{
        char c;
        if (pc.readable() && pc.read(&c, 1) == 1)
                return c;
        return -1;
}
```

2. Paste in SETUP (before while)

```
pc.set_blocking(false);
pc_print("Type a key\r\n");
```

3. Use it in the LOOP (example)

```
int key = pc_getc();
if (key == '1')
        pc_print("you typed 1\r\n");
```

> **Adjust**
>
> Don't use printf in the same program (this owns USBTX/USBRX). Use pc_print instead.
>
> pc_getc() returns -1 when nothing was typed.

M12 - PWM: RGB brightness and speaker beep

Dim the RGB LED (0.0 ~ 1.0) and play a tone on the speaker. Use this INSTEAD of M02 (same pins).

1. Paste ABOVE main()

```
PwmOut pwm_r(p23), pwm_g(p24), pwm_b(p25);      // RGB (active low)
PwmOut speaker(p26);

// each colour 0.0 ~ 1.0
void set_rgb_pwm(float r, float g, float b)
{
        pwm_r = 1.0f - r;
        pwm_g = 1.0f - g;
        pwm_b = 1.0f - b;
}

void beep(int freq, int ms)
{
        speaker.period(1.0f / freq);
        speaker = 0.5f;
        ThisThread::sleep_for(std::chrono::milliseconds(ms));
        speaker = 0.0f;
        pwm_r.period_ms(10);                    // shared period back for the RGB
}
```

2. Paste in SETUP (before while)

```
pwm_r.period_ms(10);
set_rgb_pwm(0, 0, 0);                   // off
```

3. Use it in the LOOP (example)

```
set_rgb_pwm(0.5f, 0, 0);                // half red
```

> **Adjust**
>
> brightness 0.0 = off, 1.0 = full.
>
> beep(440, 200) = 440 Hz for 200 ms (blocks for that time).
>
> All PWM pins share one period: beep() sets it back to 10 ms for the RGB.

M13 - Button interrupt + debounce + Timeout

Count joystick-centre presses by interrupt (debounced), and switch an LED off 1 s later with a Timeout.

1. Paste ABOVE main()

```
InterruptIn btn(p14);                   // joystick centre
Timer       debounce;
Timeout     flash_timeout;
DigitalOut  flash_led(LED4);

volatile int  presses = 0;
volatile bool press_flag = false;

void flash_off() { flash_led = 0; }

void on_press()                         // interrupt: keep it short
{
        if (debounce.elapsed_time() < 200ms)    // ignore switch bounce
                return;
        debounce.reset();
        presses++;
        press_flag = true;
        flash_led = 1;
        flash_timeout.attach(&flash_off, 1s);
}
```

2. Paste in SETUP (before while)

```
debounce.start();
btn.mode(PullDown);
btn.rise(&on_press);
```

3. Use it in the LOOP (example)

```
if (press_flag)
{
        press_flag = false;
        printf("pressed %d times\r\n", presses);
}
```

> **Adjust**
>
> Read press_flag in main (and set it back to false) to react to a press with printf / LCD.
>
> Change p14 to another joystick pin for another button.

M14 - UART between two boards (text lines)

Send a text line to the other board and receive its lines. Wire p9 -> other p10, p10 <- other p9, GND - GND.

1. Paste ABOVE main()

```
BufferedSerial link(p9, p10, 9600);     // TX, RX
char rx_line[32];
int  rx_len = 0;

void link_send(const char *s) { link.write(s, strlen(s)); }

// call every loop; true when a whole line arrived (now in rx_line)
bool link_receive_line()
{
        char c;
        while (link.read(&c, 1) == 1)
        {
                if (c == '\n') {
                        rx_line[rx_len] = 0;
                        rx_len = 0;
                        return true;
                }
                if (rx_len < 30)
                        rx_line[rx_len++] = c;
        }
        return false;
}
```

2. Paste in SETUP (before while)

```
link.set_blocking(false);
```

3. Use it in the LOOP (example)

```
link_send("HELLO\n");
if (link_receive_line()) {
        // rx_line holds the text, e.g. compare: if (strcmp(rx_line, "HELLO") == 0) ...
}
```

> **Adjust**
>
> Both boards: same baud (9600).
>
> End every message with \n.
>
> link_receive_line() returns true when a full line is in rx_line.

M15 - RTOS: a thread and a mutex

Run a job in its own thread at its own rate. Protect shared numbers with a Mutex.

1. Paste ABOVE main()

```
Thread worker(osPriorityNormal, 2048);
Mutex  data_mutex;
int    shared_value = 0;                        // used by worker and main

void worker_task()
{
        while (true)
        {
                data_mutex.lock();
                shared_value++;                 // ---- your job here ----
                data_mutex.unlock();
                ThisThread::sleep_for(100ms);
        }
}
```

2. Paste in SETUP (before while)

```
worker.start(worker_task);
```

3. Use it in the LOOP (example)

```
data_mutex.lock();
int v = shared_value;                   // take a copy, then unlock
data_mutex.unlock();
```

> **Adjust**
>
> Copy the thread block for more threads (new names).
>
> Stack 2048 if the thread uses printf / sprintf.
>
> Every thread loop needs a sleep_for.

M16 - RTOS: EventQueue (interrupt -> printf / LCD)

Let an interrupt or a timer run code that is allowed to printf and draw, by posting it to an EventQueue thread.

1. Paste ABOVE main()

```
EventQueue queue(32 * EVENTS_EVENT_SIZE);
Thread     queue_thread(osPriorityNormal, 4096);
InterruptIn q_btn(p15);                 // joystick up

void every_second() { printf("1 s\r\n"); }              // runs in queue_thread
void on_up_job()    { printf("up pressed\r\n"); }
void on_up_isr()    { queue.call(on_up_job); }          // interrupt: just post
```

2. Paste in SETUP (before while)

```
q_btn.mode(PullDown);
q_btn.rise(&on_up_isr);
queue.call_every(1s, every_second);
queue_thread.start(callback(&queue, &EventQueue::dispatch_forever));
```

> **Adjust**
>
> Put slow work in the functions run by the queue.
>
> call_every replaces a Ticker; call_in replaces a Timeout.

Putting modules together (example 18)

POT1 sets the speed of the running LEDs, and the LCD shows the speed and a bar. It is just M01 + M04 + M05 + M07 + M08 + M09 pasted in their places, plus 10 lines in the loop. Compiled with 0 errors.

```
//
// Example 18 : five code modules put together (section 15)
//
//   POT1 sets the speed of a running light on LED1 ~ LED4 (100 ~ 1000 ms)
//   LCD line 1 : "Speed 450 ms", line 2 : a bar for POT1, line 3 : LED number
//   Modules used : M01 LEDs, M04 LCD text, M05 bar, M07 Timer task,
//                  M08 running LEDs, M09 pots
//
#include "mbed.h"
#include "lcd_c12832.h"

// ---- M01 : LED1 ~ LED4 ----
DigitalOut led[4] = { DigitalOut(LED1), DigitalOut(LED2), DigitalOut(LED3), DigitalOut(LED4) };

void led_one(int n)
{
        for (int i = 0; i < 4; i++)
                led[i] = (i == n);
}

// ---- M04 : LCD ----
LCD_C12832 lcd;
char lcd_text[24];

// ---- M05 : bar that grows right ----
void lcd_hbar(int y, int value, int max)
{
        lcd.box(0, y, 128, 6);
        lcd.fill_box(1, y + 1, value * 126 / max, 4);
}

// ---- M07 : Timer task ----
Timer task_timer;
int task_ms = 300;

// ---- M08 : running LEDs ----
int run_pos = 0;

void run_forward() { run_pos = (run_pos + 1) % 4;  led_one(run_pos); }

// ---- M09 : pots ----
AnalogIn pot1(p19);

int read_pot(AnalogIn &pot)
{
        int sum = 0;
        for (int i = 0; i < 8; i++)
                sum += pot.read_u16() >> 4;
        return sum / 8;
}

int map_pot(int raw, int lo, int hi) { return lo + (hi - lo) * raw / 4095; }

int main()
{
        lcd.init();                             // M04 setup
        task_timer.start();                     // M07 setup

        while (true)
        {
                int p1 = read_pot(pot1);                // M09
                task_ms = map_pot(p1, 1000, 100);       // pot up = faster

                if (task_timer.elapsed_time() >= std::chrono::milliseconds(task_ms))    // M07
                {
                        task_timer.reset();
                        run_forward();                  // M08
                }
```

```
                lcd.clear();                            // M04 + M05
                sprintf(lcd_text, "Speed %d ms", task_ms);
                lcd.text(0, 0, lcd_text);
                lcd_hbar(11, p1, 4095);
                sprintf(lcd_text, "LED%d on", run_pos + 1);
                lcd.text(0, 22, lcd_text);
                lcd.update();

                ThisThread::sleep_for(20ms);
        }
}
```

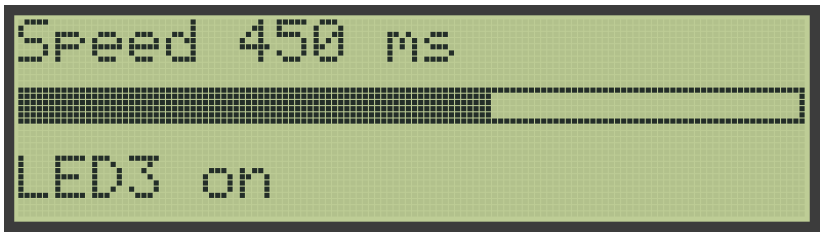


Figure - LCD of example 18 with POT1 at about 60 %: speed 450 ms, LED3 lit.